

Chapter 1

LLM Architecture and Optimization Methods

This section covers the foundational architecture of large language models and the key optimization techniques that make training and inference efficient. Topics are ordered as a curriculum: we begin with the transformer itself, then cover how to train it efficiently, how to adapt it cheaply, how to compress it, how to scale it, and how to accelerate its inference.

1.1 How LLMs Work: An Intuitive Overview

Before diving into architectural details, let us build intuition for how a large language model transforms text into text. The entire process follows a simple pipeline: **text** $\rightarrow$ **tokens** $\rightarrow$ **representations** $\rightarrow$ **tokens** $\rightarrow$ **text**.

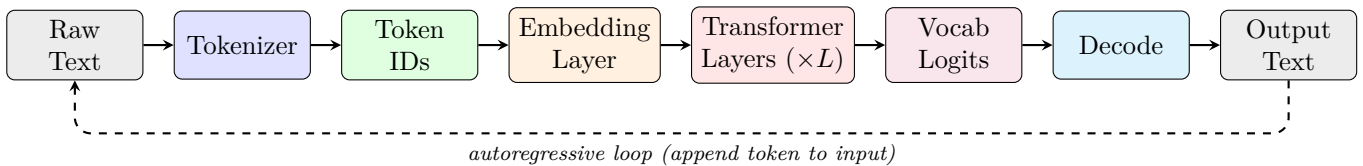


Figure 1.1: The LLM pipeline: text is tokenized into subword units, converted to integer IDs, embedded as dense vectors, processed through transformer layers, projected to vocabulary logits, and decoded back to text. The dashed loop shows autoregressive generation—each output token is appended to the input for the next forward pass.

The Four Key Stages

1. **Tokenization:** Raw text is split into subword pieces (not characters, not full words) using a learned vocabulary. “unhappiness” might become [“un”, “happiness”] or [“unhapp”, “iness”].
2. **Embedding:** Each token ID indexes into a learned embedding table, producing a dense vector in $\mathbb{R}^d$ (typically $d = 4096$). These vectors capture semantic meaning—similar words get similar vectors.
3. **Contextual Processing:** The transformer stack processes all embeddings in parallel, using self-attention to let each position “read” from all other positions. After L layers, each position’s hidden state encodes rich contextual information.
4. **Prediction:** The final hidden state is projected to a probability distribution over the full vocabulary, and a decoding strategy selects the next token.

1.2 Tokenization

Tokenization is the critical first step that converts raw text into the discrete symbols a language model operates on. The choice of tokenizer directly affects model quality, multilingual capability, and computational efficiency.

Why Subwords?

Character-level models need very long sequences (expensive attention). Word-level models cannot handle rare or novel words. Subword tokenization strikes the ideal balance: common words are single tokens (“the” → [the]), rare words decompose into known pieces (“cryptocurrency” → [“crypt”, “ocur”, “rency”]), and the vocabulary stays manageable (32K–128K tokens).

1.2.1 Why Not Characters or Words?**Table 1.1:** Trade-offs of different tokenization granularities.

Granularity	Vocab Size	Seq Length	Issues
Character	~256	Very long	Attention cost $O(n^2)$; hard to learn long-range semantics
Word	~500K+	Short	Cannot handle rare/novel words; huge embedding table
Subword	32K–128K	Moderate	Best trade-off: short sequences, open vocabulary

1.2.2 Byte-Pair Encoding (BPE)

BPE [320] is the dominant tokenization algorithm used by GPT, Llama, Mistral, and most modern LLMs.

BPE Algorithm

1. Start with a vocabulary of individual characters (bytes)
2. Count all adjacent symbol pairs in the training corpus
3. Merge the most frequent pair into a new symbol
4. Repeat steps 2–3 for k iterations (until desired vocabulary size)

**Figure 1.2:** BPE tokenization example: starting from characters, the algorithm iteratively merges the most frequent adjacent pairs until the word becomes a single token or the vocabulary budget is exhausted.

1.2.3 Other Tokenization Methods

Table 1.2: Comparison of subword tokenization algorithms.

Method	Used By	Key Idea
BPE	GPT-4 [274], Llama-3 [118], Mistral [165]	Bottom-up merging of frequent pairs; deterministic
WordPiece	BERT [76], Distil-BERT [312]	Similar to BPE but maximizes likelihood of training data
Unigram LM	SentencePiece (T5 [303], XLNet [406])	Top-down: start with large vocab, prune by likelihood impact
Byte-level BPE	GPT-2 [301]+	BPE on raw bytes (no unknown tokens possible); 256 base vocab

1.2.4 Tokenization Best Practices

1. **Vocabulary size matters:** 32K is minimal; 128K enables better multilingual coverage and code handling. Llama-3 uses 128K tokens.
2. **Special tokens:** Always include <bos>, <eos>, <pad>, <unk>. For instruction-tuned models, add role markers (<|user|>, <|assistant|>).
3. **Fertility:** Measure tokens-per-word across languages. High fertility (many tokens per word) indicates poor coverage for that language.
4. **Never tokenize across boundaries:** Spaces, punctuation, and digits should be handled consistently. Most modern tokenizers prepend a space marker (“the”) to distinguish word-initial vs. continuation tokens.
5. **Numbers:** Consider digit-level tokenization for arithmetic tasks. “2024” as [“2”, “0”, “2”, “4”] enables digit-by-digit reasoning.
6. **Code:** Ensure whitespace (indentation) is tokenized efficiently. Llama-3 tokenizes runs of spaces as single tokens.

1.2.5 Tokenization in Practice: HuggingFace Example

The `transformers` library provides a unified interface for all tokenizers. The following demonstrates encoding and decoding with a modern LLM tokenizer:

```
from transformers import AutoTokenizer

# Load Llama-3 tokenizer (128K vocabulary, byte-level BPE)
tokenizer = AutoTokenizer.from_pretrained("meta-llama/Meta-Llama-3-8B")

text = "Reinforcement learning optimizes long-term rewards."

# Encode: text -> token IDs
token_ids = tokenizer.encode(text)
print(token_ids)
# [128000, 29934, 262, 11008, 4815, 6900, 1317, 9860, 21845, 13]

# Decode individual tokens to see subword splits
tokens = tokenizer.convert_ids_to_tokens(token_ids)
print(tokens)
# ['<|begin_of_text|>', 'Re', 'inforce', 'ment', ' learning',
#  ' optimizes', ' long', '-term', ' rewards', '.']

# Decode back to text (round-trip)
reconstructed = tokenizer.decode(token_ids, skip_special_tokens=True)
```

```

assert reconstructed == text # Perfect reconstruction

# Tokenize with attention mask (for batched inputs with padding)
batch = tokenizer(
    ["Short text.", "A much longer input sentence for comparison."],
    padding=True, return_tensors="pt"
)
print(batch.keys()) # dict_keys(['input_ids', 'attention_mask'])

```

Listing 1.1: Tokenization encode/decode with HuggingFace Transformers.

1.2.6 Special Tokens and Structured Prompts

Special tokens are reserved vocabulary entries that carry structural meaning rather than linguistic content. They are critical for controlling model behavior.

Table 1.3: Common special tokens across LLM families.

Token	Alias	Purpose
<bos> / < begin_of_text >	BOS	Marks start of sequence
<eos> / < end_of_text >	EOS	Marks end of sequence; stops generation
< user >	—	Marks start of user turn in chat
< assistant >	—	Marks start of assistant turn in chat
<pad>	PAD	Fills batch to uniform length; masked in attention
<unk>	UNK	Out-of-vocabulary placeholder (rare with BPE)
[SEP]	SEP	Separates segments (BERT-style)
[CLS]	CLS	Classification token (BERT)
[MASK]	MASK	Masked token for MLM pretraining

Role Markers for Instruction-Tuned Models. Modern chat models use special tokens to delineate conversational structure. These are **not** trained to carry semantic meaning—they are structural delimiters that the model learns to parse:

```

# Llama-3 chat template
messages = [
    {"role": "system", "content": "You are a helpful assistant."},
    {"role": "user", "content": "Explain PPO in one sentence."},
]

# apply_chat_template handles all special token insertion
prompt = tokenizer.apply_chat_template(messages, tokenize=False)
print(prompt)
# <|begin_of_text|><|start_header_id|>system<|end_header_id|>
#
# You are a helpful assistant.<|eot_id|><|start_header_id|>user<|end_header_id|>
#
# Explain PPO in one sentence.<|eot_id|><|start_header_id|>assistant<|
#   end_header_id|>
#
#

```

Listing 1.2: Chat template with special tokens (Llama-3 format).

Special Token Best Practices

- **Never split special tokens:** They must be atomic—ensure your tokenizer treats them as single units, not character sequences.
- **Mask loss on special tokens:** During SFT, do not compute loss on structural tokens (role markers, separators). The model should not “learn” to predict formatting.

- **Use templates for structure:** Encode task semantics via special tokens rather than natural language instructions. E.g., `<|tool_call|>` is more reliable than “Now I will call a tool.”
- **Tool/function calling:** Define dedicated tokens like `<|function|>`, `<|result|>` to create unambiguous boundaries between reasoning and action.
- **Consistent handling in RL:** During PPO/GRPO, ensure the reference model and policy model use identical tokenization and special token handling—mismatches corrupt KL computation.
- **EOS handling:** During generation, ensure EOS is included in the action space. If the model cannot emit EOS, responses grow unbounded (common RL failure mode).

1.3 The Transformer Architecture

The Transformer [357] is the foundation of all modern LLMs. Understanding its components is essential for grasping every optimization and training method in this guide.

1.3.1 High-Level Structure

A decoder-only transformer processes tokens sequentially through embedding, repeated attention+FFN blocks, and a final projection to vocabulary logits. Figure 1.3 shows the complete architecture.

1.3.2 The Original Encoder-Decoder Transformer

The Transformer was originally introduced [357] as an **encoder-decoder** architecture for sequence-to-sequence tasks (machine translation, summarization). While modern LLMs predominantly use decoder-only variants (GPT-style), understanding the full architecture is essential because cross-attention and masked self-attention — both originating here — remain fundamental building blocks.

Encoder. The encoder processes the entire input sequence *bidirectionally* — each token attends to all other tokens (no causal mask). This produces a rich contextual representation $\mathbf{H}^{\text{enc}} \in \mathbb{R}^{n \times d}$ where each position encodes information about the full input:

- **Input:** Token embeddings + sinusoidal positional encodings
- **Each layer:** Multi-Head Self-Attention $\rightarrow$ Add & Norm $\rightarrow$ FFN $\rightarrow$ Add & Norm
- **No causal mask:** Position i attends to all positions $1, \dots, n$
- **Output:** Contextual representations of the full input sequence

Decoder — Masked Multi-Head Self-Attention. The decoder generates output tokens one at a time (autoregressively). To prevent the model from “seeing the future,” the self-attention in the decoder uses a **causal mask**:

$$\text{MaskedAttn}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}} + M\right) V \quad (1.1)$$

where the mask M is:

$$M_{ij} = \begin{cases} 0 & \text{if } i \geq j \text{ (can attend)} \\ -\infty & \text{if } i < j \text{ (future token — blocked)} \end{cases}$$

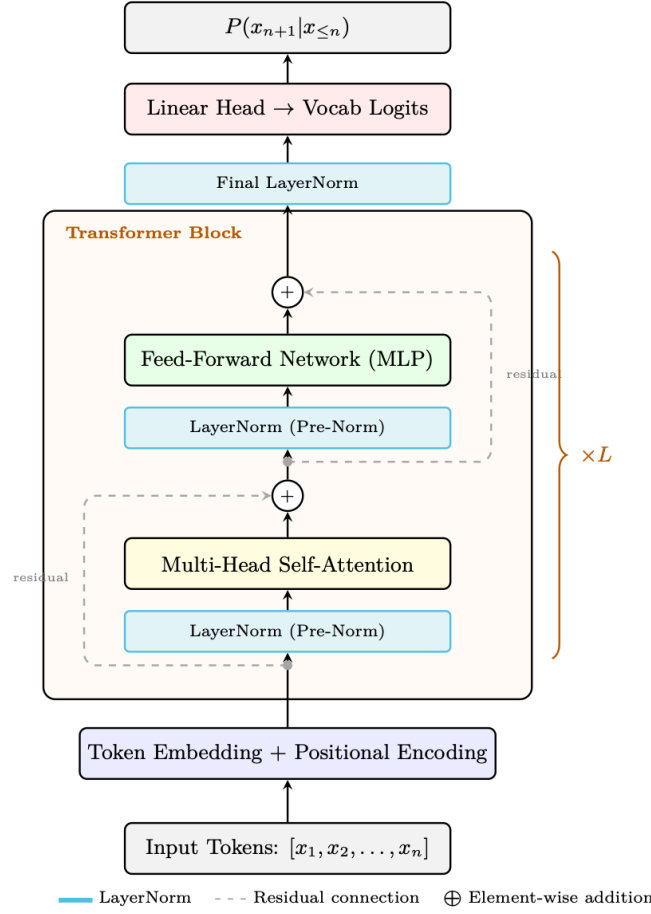


Figure 1.3: Decoder-only Transformer block (GPT-style, Pre-Norm variant). Each sub-layer (attention, FFN) is preceded by LayerNorm and followed by a residual addition: $\mathbf{x} + \text{SubLayer}(\text{LN}(\mathbf{x}))$. This Pre-Norm ordering (used by Llama, GPT-3, Mistral) stabilizes training without warmup, unlike the original Post-Norm (which applies LayerNorm after the addition). L identical blocks are stacked, followed by a final LayerNorm and linear projection to vocabulary logits.

Why Masking Matters

During training, the decoder processes the entire target sequence in parallel (teacher forcing), but each position must only attend to previous positions to maintain the autoregressive property. The mask ensures that generating token t uses only information from tokens $1, \dots, t-1$. At inference, tokens are generated one-by-one so the mask is implicit — but during training it enables parallel computation while preserving causality.

Decoder — Cross-Attention. After masked self-attention, each decoder layer applies **cross-attention** where the decoder attends to the encoder’s output representations. This is the mechanism by which the decoder “reads” the input:

$$\text{CrossAttn}(Q_{\text{dec}}, K_{\text{enc}}, V_{\text{enc}}) = \text{softmax}\left(\frac{Q_{\text{dec}}K_{\text{enc}}^T}{\sqrt{d_k}}\right)V_{\text{enc}} \quad (1.2)$$

- **Queries** come from the decoder’s previous sublayer (the masked self-attention output)
- **Keys and Values** come from the encoder’s final output $\mathbf{H}^{\text{enc}}$
- **No mask** is applied — every decoder position can attend to every encoder position
- This allows the decoder to dynamically focus on different parts of the input at each generation step (e.g., attending to “cat” when translating to “gato” in English→Spanish)

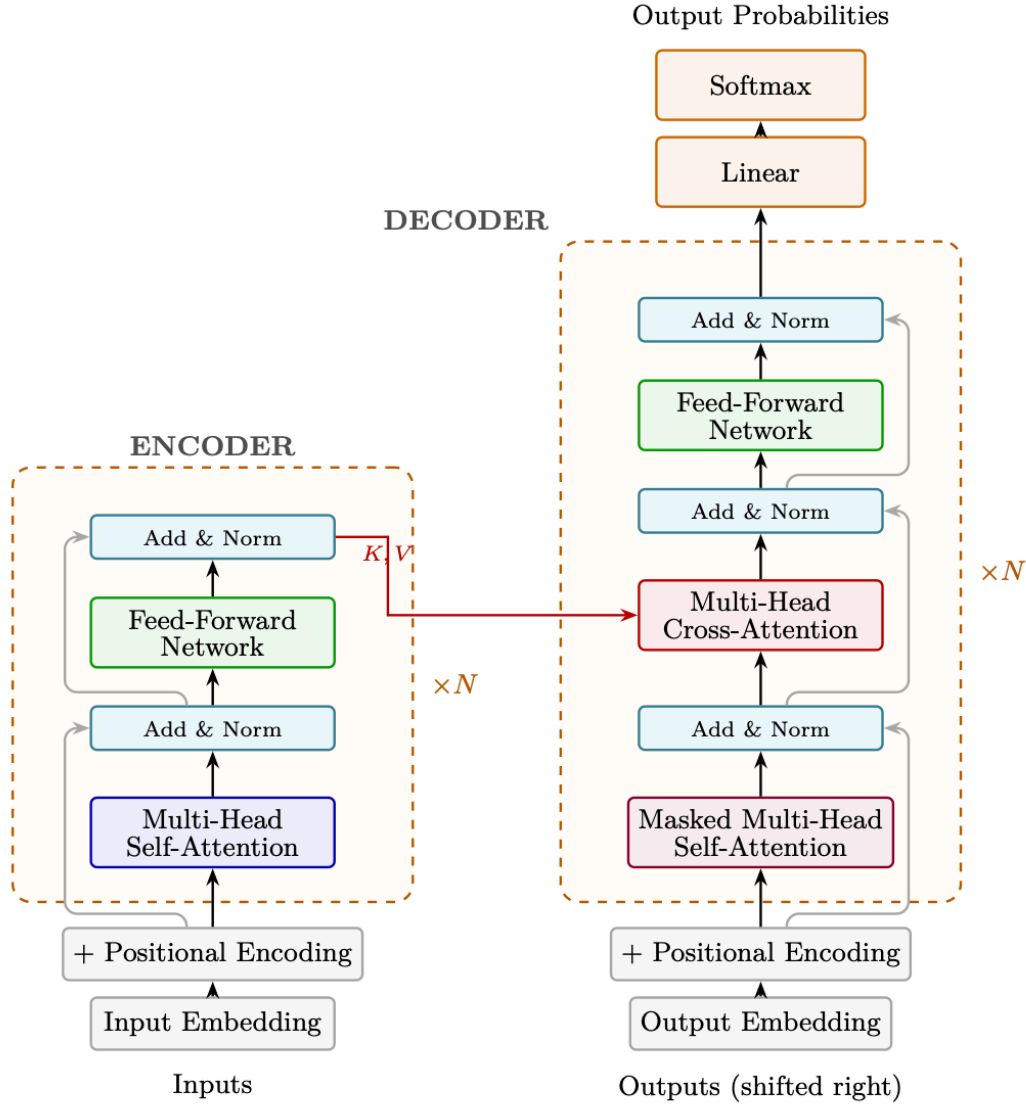


Figure 1.4: The original Transformer architecture (Vaswani et al., 2017). The encoder (left) processes the full input with bidirectional self-attention. The decoder (right) generates tokens autoregressively using masked self-attention and cross-attention to encoder representations. Dashed boxes indicate the repeated layer block ($\times N$); gray lines show residual connections bypassing each sub-layer. Note: the original work uses **Post-Norm** (LayerNorm applied *after* the residual addition: $\text{LN}(\mathbf{x} + \text{SubLayer}(\mathbf{x}))$), unlike modern LLMs which use Pre-Norm.

Full Decoder Layer. Each decoder layer contains three sublayers (vs. two in the encoder):

1. **Masked Multi-Head Self-Attention** + Residual + LayerNorm
2. **Multi-Head Cross-Attention** (to encoder output) + Residual + LayerNorm
3. **Feed-Forward Network** + Residual + LayerNorm

From Encoder-Decoder to Decoder-Only. Modern LLMs (GPT, Llama, Qwen) use only the decoder, removing both the encoder and cross-attention layers entirely. The key insight: for generative language modeling, a single causal (masked) self-attention stack is sufficient — the model learns to encode context and generate continuations in a single pass. This simplifies architecture, training, and inference while scaling more effectively. Encoder-decoder models (T5, BART) remain relevant for tasks with distinct input/output structure (translation, summarization), and cross-attention reappears in multimodal models where vision encoders provide keys/values to language decoders.

1.3.3 Decoder-Only vs Encoder-Decoder

Modern LLMs almost exclusively use decoder-only architectures, but understanding the trade-offs with encoder-decoder designs clarifies why.

Architecture	Examples	Use Case
Decoder-only	GPT-4 [274], Llama [118], Mistral [165], Qwen [350]	Autoregressive generation; dominant for chat/reasoning
Encoder-decoder	T5 [303], BART [207], Flan-T5 [59]	Seq2seq (translation, summarization); less common now
Encoder-only	BERT [76], RoBERTa [233]	Classification/embeddings; not for generation

Why Decoder-Only Won

Decoder-only models are simpler (one model, one loss), scale better (all parameters contribute to generation), and support unified training (pretraining = next-token prediction = fine-tuning objective). Encoder-decoder models waste capacity on the encoder for pure generation tasks.

1.3.4 Embeddings: From Discrete Tokens to Continuous Space

Before any attention or computation happens, the transformer must convert discrete token IDs into continuous vectors that neural networks can process. This is the role of the **embedding layer**.

What is an Embedding? An embedding is a learned dense vector representation of a discrete symbol. Instead of representing the word “king” as a one-hot vector of size $|\mathcal{V}| = 128,000$ (mostly zeros), we represent it as a compact vector in $\mathbb{R}^d$ (e.g., $d = 4096$) that captures its *meaning*.

The key insight: **similar concepts get nearby vectors**. In a well-trained embedding space:

- “king” and “queen” are close (both royalty)
- “king” and “bicycle” are far apart (unrelated)
- Vector arithmetic captures relationships: $\vec{\text{king}} - \vec{\text{man}} + \vec{\text{woman}} \approx \vec{\text{queen}}$

The Embedding Table. In practice, the embedding layer is simply a matrix $\mathbf{E} \in \mathbb{R}^{|\mathcal{V}| \times d}$ where row i stores the embedding vector for token i :

$$\text{embed}(x_t) = \mathbf{E}[x_t] \in \mathbb{R}^d \quad (1.3)$$

For a sequence of token IDs $[x_1, x_2, \dots, x_n]$, embedding is a simple table lookup (indexing operation):

$$\mathbf{H}_0 = [\mathbf{E}[x_1]; \mathbf{E}[x_2]; \dots; \mathbf{E}[x_n]] \in \mathbb{R}^{n \times d}$$

Embedding Table in Transformers

- **Size:** $|\mathcal{V}| \times d$. For Llama-3: $128,256 \times 4,096 = 525\text{M}$ parameters (6.5% of 8B model).
- **Initialization:** Random (Xavier/normal), then learned via backpropagation.
- **Weight tying:** Many models *share* the embedding matrix with the output projection head: $W_{\text{head}} = \mathbf{E}^T$. This saves parameters and creates a symmetric encode-decode structure.
- **Input:** Token ID (integer) $\rightarrow$ **Output:** Dense vector in $\mathbb{R}^d$.
- **Gradient flow:** During training, only the rows corresponding to tokens in the current batch receive gradient updates (sparse update).

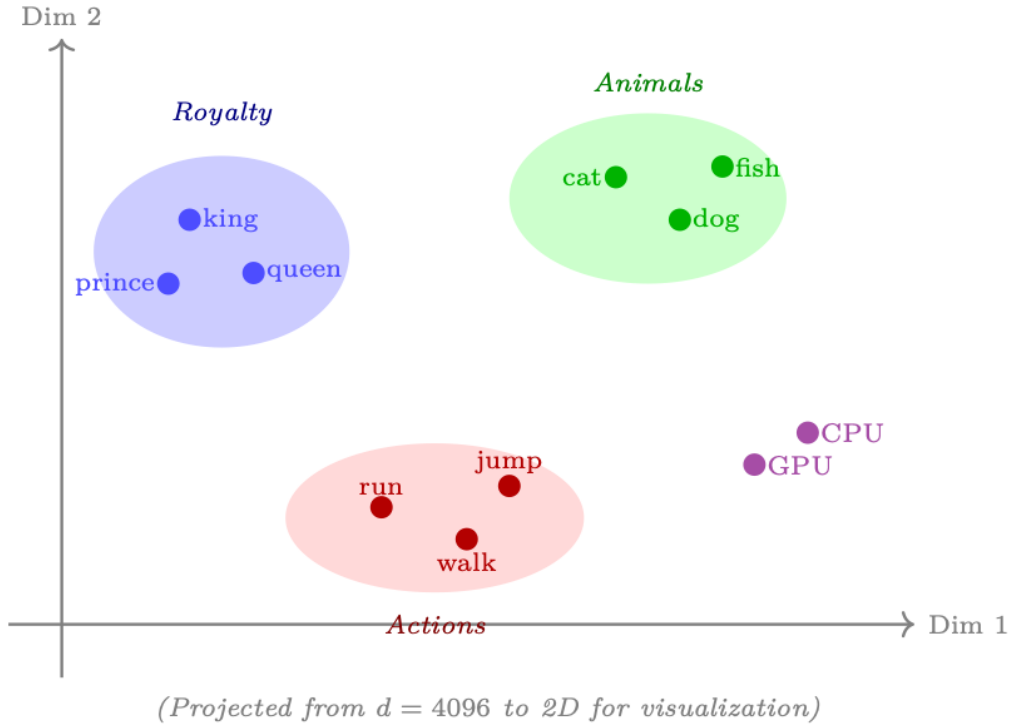


Figure 1.5: Embedding space visualization (2D projection): semantically similar words cluster together. The embedding table learns these positions during pretraining, capturing meaning purely from co-occurrence patterns in text.

Why Embeddings Work

The embedding table is learned end-to-end with the rest of the model. Because the model is trained to predict the next token, it must learn representations where tokens that appear in similar contexts get similar vectors. This is the distributional hypothesis: “you shall know a word by the company it keeps” [91]. The embedding layer compresses this statistical structure into dense geometry.

The Anisotropy Problem. A critical issue arises when using pretrained embeddings (e.g., from BERT or GPT-2) for downstream tasks like retrieval (RAG) or bootstrapping recommender systems: the learned representations are highly **anisotropic**—they occupy a narrow cone in the embedding space rather than being uniformly distributed across all directions [86].

Why this matters for applications:

- **RAG / Retrieval:** If all embeddings have cosine similarity > 0.7 regardless of content, retrieval rankings become nearly random—the system cannot distinguish relevant from irrelevant passages.
- **Recommender systems:** Using pretrained LLM embeddings to represent items/users only works if the geometry preserves meaningful similarity structure.
- **Clustering:** Anisotropic embeddings collapse clusters, making it impossible to discover natural groupings.

Resolution: Whitening. A simple and effective fix is **whitening** [340]—a linear transformation that makes the embedding distribution isotropic (zero mean, identity covariance):

$$\tilde{\mathbf{h}} = \mathbf{D}^{-1/2} \mathbf{U}^T (\mathbf{h} - \boldsymbol{\mu}) \quad (1.4)$$

where $\boldsymbol{\mu}$ is the mean embedding, and $\mathbf{U} \mathbf{D} \mathbf{U}^T$ is the eigendecomposition of the covariance matrix $\Sigma = \frac{1}{N} \sum_i (\mathbf{h}_i - \boldsymbol{\mu})(\mathbf{h}_i - \boldsymbol{\mu})^T$.

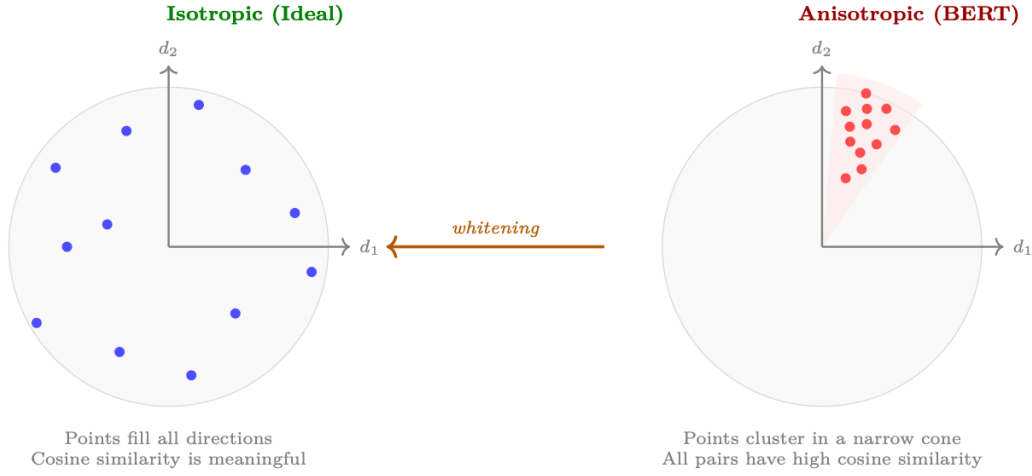


Figure 1.6: Isotropy vs. anisotropy in embedding spaces. Left: isotropic embeddings spread uniformly, making cosine similarity a reliable measure of semantic relatedness. Right: anisotropic embeddings (as found in BERT) cluster in a narrow cone, causing all pairs to have high cosine similarity regardless of semantic content. Whitening transforms the space to restore isotropy.

Whitening in Practice

- **What it does:** Rotates and scales the embedding space so all directions have equal variance (unit covariance).
- **Effect:** Cosine similarity becomes meaningful—semantically similar pairs score high, dissimilar pairs score low.
- **Bonus:** Can simultaneously reduce dimensionality by keeping only the top- k eigenvectors (similar to PCA), making retrieval faster.
- **Cost:** Requires computing the covariance matrix over a representative corpus (one-time, $O(N \cdot d^2)$). The transform itself is a simple matrix multiply at inference.
- **Alternative approaches:** Contrastive fine-tuning (SimCSE), flow-based normalization, or training with isotropy-promoting regularizers.

1.3.5 Self-Attention Mechanism

Self-attention is the core operation that allows each token to attend to every other token in the sequence, computing a weighted combination based on relevance.

Scaled Dot-Product Attention

Given input sequence $X \in \mathbb{R}^{n \times d}$, we compute:

$$Q = XW_Q, \quad K = XW_K, \quad V = XW_V \quad (W_Q, W_K, W_V \in \mathbb{R}^{d \times d_k})$$

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}} + M\right)V$$

where M is the **causal mask** (for autoregressive models): $M_{ij} = 0$ if $i \geq j$, else $-\infty$.

Intuition: Each token “attends” to all previous tokens, computing a weighted average of their values based on query-key similarity.

Computational Complexity. The naive attention computation has **quadratic cost** in sequence length:

- **Time:** $O(n^2 \cdot d)$ — computing QK^T requires n^2 dot products, each of dimension d_k .

- **Memory:** $O(n^2)$ — the full attention matrix must be materialized to apply softmax.

For a 128K-token context with $d = 4096$, the attention matrix alone is $128K \times 128K = 16.4$ billion entries (64 GB in FP32). This quadratic scaling is the fundamental bottleneck for long-context LLMs.

Table 1.4: Attention cost scaling: why naive implementation is prohibitive for long sequences.

Seq Length	Attention Ops	Matrix Size	Practical Impact
2K	4M	16 MB	Fast; fits in SRAM
8K	64M	256 MB	Manageable with FlashAttention
32K	1B	4 GB	Requires memory-efficient kernels
128K	16B	64 GB	Exceeds single GPU HBM
1M	1T	4 TB	Impossible without sub-quadratic methods

Approaches to Taming Attention Cost. Several families of solutions address this quadratic bottleneck:

1. **Exact attention with IO-awareness (FlashAttention [65]):** Does not reduce computational complexity but eliminates the need to materialize the $n \times n$ matrix in HBM by computing attention in tiles that fit in SRAM. Crucially, FlashAttention is **orthogonal** to the sparse patterns below—it is an execution engine, not an attention pattern. Production systems routinely combine FlashAttention with sliding windows or block-sparse masks, getting both IO efficiency and reduced FLOPs. We cover the algorithm in detail in Section 1.6.
2. **Sliding window / local attention:** Each token only attends to the w nearest tokens (e.g., $w = 4096$). Cost becomes $O(n \cdot w)$ —linear in n . Used by Mistral [165] (window = 4096) and Longformer [23]. Trades global context for efficiency; works well because most attention is local in practice. In modern stacks, the sliding-window mask is executed *inside* a FlashAttention kernel.
3. **Sparse attention patterns:** Combine local windows with periodic global tokens (e.g., every 512th token attends to all). BigBird [416] and LongT5 [125] use this. Preserves some long-range connectivity at $O(n\sqrt{n})$ cost. Again, FlashAttention serves as the underlying kernel for the non-zero attention blocks.
4. **Linear attention / state-space models:** Replace $\text{softmax}(QK^T)V$ with $\phi(Q)(\phi(K)^TV)$ using associativity, or reformulate as a recurrence (Mamba [122], RWKV [288]). Theoretically $O(n \cdot d^2)$ total. Unlike approaches 2–3 above, these are *architectural replacements* that alter model expressiveness—softmax-free attention is fundamentally less expressive, and empirically these models still lag behind transformers on tasks requiring precise long-range retrieval or complex reasoning.
5. **KV cache compression:** At inference, compress or evict old KV pairs to bound memory. Techniques include: H₂O [429] (heavy-hitter oracle—keep only high-attention keys), StreamingLLM [394] (keep initial “attention sink” tokens + recent window), and quantized KV caches [237].

FlashAttention + Sparse Patterns = Best of Both Worlds

A common misconception is that FlashAttention is an *alternative* to sparse attention. It is not—it is an IO optimization for the attention kernel that composes freely with any attention mask. Modern production systems (e.g., Mistral, DeepSeek) use FlashAttention as the execution engine *underneath* a sliding-window or block-sparse mask. This gives you both reduced FLOPs (from sparsity) and optimal memory access patterns (from tiling). RingAttention [221] extends this further to multi-device settings, distributing the tiled computation across GPUs along the sequence

dimension.

Linear attention and state-space models (Mamba, RWKV) are a genuinely different architectural choice—they sacrifice the full pairwise interaction for $O(n)$ compute. While theoretically elegant, they have not matched transformer quality on knowledge-intensive or long-range reasoning tasks, and frontier labs continue to use exact attention (with FlashAttention + sparsity) as the backbone.

1.3.6 Multi-Head Attention

Rather than computing a single attention function, multi-head attention runs several attention operations in parallel, each learning to focus on different aspects of the input (syntax, semantics, position, etc.).

Multi-Head Attention

Instead of one attention function with d -dimensional keys/values, use H parallel heads with dimension $d_k = d/H$:

$$\text{MultiHead}(X) = \text{Concat}(\text{head}_1, \dots, \text{head}_H)W_O$$

Each head can learn different attention patterns (e.g., one head for syntax, another for semantics, another for positional proximity).

Grouped Query Attention (GQA): Llama-3 [118] uses fewer K,V heads than Q heads (e.g., 8 KV heads shared across 32 Q heads). This reduces KV cache size by $4\times$ with minimal quality loss.

Multi-head Latent Attention (MLA). DeepSeek-V2 [69] introduced a more aggressive compression: instead of sharing KV heads (as in GQA), MLA compresses *all* key-value information into a single low-rank latent vector per token:

$$c_t = W_{DKV} h_t, \quad K_t = W_{UK} c_t, \quad V_t = W_{UV} c_t$$

where $c_t \in \mathbb{R}^{d_c}$ with $d_c \ll n_h \cdot d_h$. The KV cache stores only c_t —smaller than even GQA—while decompression matrices W_{UK}, W_{UV} reconstruct full keys and values on-the-fly. Because the compression is learned end-to-end, quality matches or exceeds GQA. MLA enables DeepSeek’s subsequent Sparse Attention for sub-quadratic long-context scaling and is now the basis of the DeepSeek-V3/V4 architecture family.

1.3.7 Positional Encodings

Transformers are permutation-equivariant by construction — without positional information, the model cannot distinguish “the cat sat on the mat” from “mat the on sat cat the”. Positional encodings inject sequence-order signal so that attention can reason about token distance and direction.

Table 1.5: Positional encoding methods in modern LLMs.

Method	Used By	Key Idea
Sinusoidal	Original Transformer	Fixed sin / cos at different frequencies. Not learned.
Learned Absolute	GPT-2 [301], BERT [76]	Learned embedding per position. Limited to training length.
RoPE (Rotary)	Llama [118], Qwen [350], Mistral [165]	Rotate Q,K vectors by position-dependent angle. Extrapolates via NTK-aware scaling.
ALiBi	BLOOM [383], MPT [260]	No position embedding; add linear bias $-m i - j $ to attention scores. Simple, extrapolates well.

Sinusoidal (Fixed) Positional Encoding. Introduced in the original Transformer [357], this method uses fixed sinusoidal functions at geometrically-spaced frequencies:

$$\text{PE}(\text{pos}, 2i) = \sin\left(\frac{\text{pos}}{10000^{2i/d}}\right), \quad \text{PE}(\text{pos}, 2i+1) = \cos\left(\frac{\text{pos}}{10000^{2i/d}}\right)$$

where pos is the token position, i is the dimension index, and d is the model dimension.

Motivation: Each frequency encodes position at a different scale (analogous to binary counting). The authors hypothesised that the model could learn to attend to relative positions because $\text{PE}(\text{pos}+k)$ can be expressed as a linear function of $\text{PE}(\text{pos})$.

Pros: Zero learned parameters; deterministic; theoretically supports arbitrary lengths.

Cons: In practice, does not extrapolate well beyond training lengths; the model must learn to decode relative position from absolute signals indirectly; largely superseded.

Learned Absolute Positional Embedding. Used by GPT-2 [301] and BERT [76]: a learnable embedding matrix $\mathbf{E}_{\text{pos}} \in \mathbb{R}^{L_{\text{max}} \times d}$ is added to token embeddings:

$$h_0^{(\text{pos})} = \text{TokenEmbed}(x_{\text{pos}}) + \mathbf{E}_{\text{pos}}[\text{pos}]$$

Motivation: Let the model learn whatever positional representation is optimal for the task, rather than imposing a fixed structure.

Pros: Maximum flexibility; simple implementation; often outperforms sinusoidal for short sequences.

Cons: Hard-coded maximum length L_{max} ; no generalisation beyond it; embeddings near the end of L_{max} are under-trained; adds $L_{\text{max}} \times d$ parameters.

Rotary Position Embedding (RoPE). RoPE [341] encodes position by *rotating* query and key vectors in 2D subspaces:

$$\text{RoPE}(x_m, m) = \begin{pmatrix} x_m^{(1)} \\ x_m^{(2)} \\ \vdots \\ x_m^{(d-1)} \\ x_m^{(d)} \end{pmatrix} \odot \begin{pmatrix} \cos m\theta_1 \\ \cos m\theta_1 \\ \vdots \\ \cos m\theta_{d/2} \\ \cos m\theta_{d/2} \end{pmatrix} + \begin{pmatrix} -x_m^{(2)} \\ x_m^{(1)} \\ \vdots \\ -x_m^{(d)} \\ x_m^{(d-1)} \end{pmatrix} \odot \begin{pmatrix} \sin m\theta_1 \\ \sin m\theta_1 \\ \vdots \\ \sin m\theta_{d/2} \\ \sin m\theta_{d/2} \end{pmatrix}$$

where $\theta_i = 10000^{-2i/d}$ and m is the position index. The key property is that the dot product between rotated queries and keys depends only on relative position:

$$\langle \text{RoPE}(q_m, m), \text{RoPE}(k_n, n) \rangle = f(q_m, k_n, m - n)$$

Motivation: Achieve relative position encoding without explicit bias terms, while maintaining compatibility with linear attention and KV-caching.

Pros: Naturally relative; no extra parameters; compatible with efficient inference; can be extended to longer contexts via NTK-aware scaling [289] or YaRN (adjusting θ base or interpolating frequencies).

Cons: Slightly more compute per attention operation (rotation + interleaving); extrapolation requires explicit scaling strategies; rotation in 2D subspaces imposes structure that may not be optimal for all tasks.

RoPE Length Extension

To extend a RoPE model trained at L to context length $L' > L$:

- **Position interpolation:** Scale positions by L/L' so all positions fit in $[0, L]$. Simple but

compresses resolution.

- **NTK-aware scaling:** Increase the θ base (e.g. $10000 \rightarrow 10000 \cdot (L'/L)^{d/(d-2)}$), effectively stretching high-frequency components while preserving low-frequency ones.
- **YaRN [289]:** Combines NTK scaling with an attention temperature correction $t = 0.1 \ln(s)+1$ to compensate for increased entropy at longer distances.

ALiBi (Attention with Linear Biases). ALiBi [292] takes a radically different approach: *no positional embedding at all*. Instead, a static linear penalty is subtracted from attention scores:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}} - m \cdot [|i - j|]_{i,j}\right) V$$

where m is a head-specific slope (set geometrically: $m_h = 2^{-8h/H}$ for head h of H total). The bias $-m|i - j|$ creates a soft local attention window whose width varies by head.

Motivation: Position should bias attention toward nearby tokens (recency prior) without interfering with the embedding space. By operating purely in attention-score space, ALiBi avoids polluting token representations with positional signal.

Pros: Excellent length extrapolation (trained at 1k, works at 8k+); zero parameters; trivial to implement; head-specific slopes give multi-scale locality.

Cons: Less expressive for tasks requiring precise long-range positional reasoning (e.g. “what was the 5th word?”); the linear decay is a strong inductive bias that may not suit all domains; largely overtaken by RoPE in recent models due to RoPE’s better short-context performance.

Table 1.6: Positional encoding comparison: practical trade-offs.

	Sinusoidal	Learned Abs.	RoPE	ALiBi
Extra parameters	None	$L_{\max} \times d$	None	None
Position type	Absolute	Absolute	Relative	Relative (implicit)
Length extrapolation	Poor	None	Good (w/ scaling)	Excellent
Compute overhead	Negligible	Negligible	Small	Negligible
Dominant era	2017–19	2018–20	2022–present	2022–23

Scaling to Extremely Long Contexts (100K–1M+ Tokens). Modern frontier models (Claude [11] with 200K–1M context, Gemini 1.5 [108] at 1M+, GPT-4 [274] at 128K) require positional encodings that remain faithful far beyond training lengths. The dominant solutions today:

1. **RoPE with frequency scaling:** The standard approach for extending RoPE beyond training length. Rather than retraining, the base frequency θ is rescaled:

$$\theta'_i = \theta_i \cdot \left(\frac{L_{\text{target}}}{L_{\text{train}}}\right)^{2i/d}$$

Variants include:

- **Linear scaling** (Position Interpolation) [48]: Simply divide position indices by a factor s . Cheap but degrades quality at high extension ratios.
- **NTK-aware scaling** [289]: Scale the base frequency $\theta = 10000 \rightarrow 10000 \cdot s^{d/(d-2)}$. Preserves high-frequency (local) information while extending low-frequency (global) range.
- **YaRN** [289] (Yet another RoPE extensioN): Combines NTK scaling with an attention temperature correction and fine-tuning on a small long-context corpus. Used by Llama-3 to extend from 8K training to 128K deployment.

- **Dynamic NTK** [289]: Adjusts the scaling factor on-the-fly based on actual sequence length at inference. No fixed extension ratio needed—the model adapts as context grows.
- 2. **Continued pretraining on long data:** Even with RoPE scaling, models benefit from a short continued pretraining phase (1–5B tokens) on long documents. This teaches the model to actually *use* distant context, not just tolerate it positionally. Llama-3.1 used a progressive schedule: 8K $\rightarrow$ 64K $\rightarrow$ 128K.
- 3. **Ring Attention / Blockwise Parallel** [221]: For sequences exceeding single-GPU memory (1M+ tokens), Ring Attention distributes the sequence across GPUs in a ring topology. Each GPU holds a block and passes KV blocks around the ring, computing local attention tiles. This enables linear memory scaling with GPU count while preserving exact attention.
- 4. **Hybrid architectures:** Some systems combine a local sliding window (e.g., 4K) for most layers with full attention at select layers (e.g., every 4th layer). This provides $O(n \cdot w)$ cost for most computation while maintaining global information flow.

Long Context $\neq$ Long Context Usage

A model with 1M context length does *not* necessarily use all 1M tokens effectively. The “lost in the middle” phenomenon [226] shows that models tend to focus on the beginning and end of long contexts, underutilizing information in the middle. Effective long-context utilization requires both positional encoding support *and* training on tasks that reward long-range retrieval.

1.3.8 Feed-Forward Network (MLP)

Each transformer block contains an MLP applied independently to each position:

$$\text{FFN}(x) = W_2 \cdot \sigma(W_1 x + b_1) + b_2$$

where $W_1 \in \mathbb{R}^{d \times 4d}$, $W_2 \in \mathbb{R}^{4d \times d}$. Modern LLMs use:

- **SwiGLU activation:** $\text{FFN}(x) = W_2(\text{Swish}(W_1 x) \odot W_3 x)$ — used by Llama [118], Mistral [165]. Requires 3 weight matrices but gives better performance.
- Hidden dimension is typically $8/3 \times d$ (rounded to multiples of 256 for Tensor Core efficiency).

FFN as Memory

Recent work [110] suggests the FFN layers act as a *key-value memory*: W_1 rows are keys (patterns to match), W_2 columns are values (information to output). The FFN “retrieves” stored knowledge based on the current hidden state.

1.3.9 Layer Normalization

Layer normalization stabilizes training by normalizing activations across the feature dimension. Its placement relative to the attention/FFN sublayers significantly affects training dynamics.

How LayerNorm Works. Given a hidden state vector $\mathbf{x} \in \mathbb{R}^d$ (a single token’s representation), LayerNorm [15] computes:

$$\text{LayerNorm}(\mathbf{x}) = \gamma \odot \frac{\mathbf{x} - \mu}{\sqrt{\sigma^2 + \epsilon}} + \beta \tag{1.5}$$

where:

- $\mu = \frac{1}{d} \sum_{i=1}^d x_i$ (mean across the d feature dimensions)
- $\sigma^2 = \frac{1}{d} \sum_{i=1}^d (x_i - \mu)^2$ (variance across features)

- $\gamma, \beta \in \mathbb{R}^d$ are **learned** scale and shift parameters (per-dimension)
- $\epsilon \approx 10^{-5}$ prevents division by zero

Key distinction from BatchNorm: LayerNorm normalizes across the *feature dimension* of a single example, not across the batch. This makes it independent of batch size and works identically at training and inference.

RMSNorm — The Modern Simplification. RMSNorm [421] drops the mean-centering step, normalizing only by the root-mean-square:

$$\text{RMSNorm}(\mathbf{x}) = \gamma \odot \frac{\mathbf{x}}{\text{RMS}(\mathbf{x})}, \quad \text{RMS}(\mathbf{x}) = \sqrt{\frac{1}{d} \sum_{i=1}^d x_i^2} \quad (1.6)$$

No β (shift) parameter and no mean subtraction — just scale. This saves one reduction operation per token and is $\sim 5\text{--}10\%$ faster on GPUs while achieving equivalent model quality. All modern LLMs (Llama, Mistral, Qwen) use RMSNorm.

Pre-LN vs Post-LN

- **Post-LN** (original Transformer): $h + \text{LayerNorm}(\text{Attn}(h))$. Requires careful warmup; training can be unstable.
- **Pre-LN** (GPT-2+, all modern LLMs): $h + \text{Attn}(\text{LayerNorm}(h))$. Stabilizes training; enables higher learning rates.
- **RMSNorm** (Llama [118], Mistral [165]): Simplified LayerNorm without mean-centering: $\text{RMSNorm}(x) = x / \text{RMS}(x) \cdot \gamma$. Slightly faster, same quality.

Why Normalization Matters for Deep Networks

Without normalization, activations tend to grow or shrink exponentially through layers (exploding/vanishing activations). A 128-layer transformer without LayerNorm would see magnitudes vary by $10^{30} \times$ between the first and last layer. Normalization constrains each layer’s output to a predictable range, enabling stable gradient flow and allowing the optimizer to use consistent learning rates throughout the network.

1.3.10 Model Size Reference

The following table summarizes key architectural parameters for widely-used open-weight models (latest versions as of 2025), providing a quick reference for understanding scale and design choices.

Table 1.7: Architecture parameters for popular open-weight LLMs (2024–2025 generation).

Model	Params	Layers	d	Heads	KV Heads	Context
Llama-3.1 8B [118]	8B	32	4096	32	8	128K
Llama-3.1 405B [118]	405B	126	16384	128	8	128K
Llama-4 Maverick [4]	400B (17B active)	48	5120	40	8	1M
Mistral Large 2 [5]	123B	88	12288	96	8	128K
Qwen-2.5 72B [350]	72B	80	8192	64	8	128K
DeepSeek-V3 [70]	671B (37B active)	61	7168	128	MLA	128K

Note: Models marked with “active” parameters use Mixture-of-Experts (MoE) architecture—total parameters indicate model capacity, while active parameters reflect per-token compute cost. DeepSeek-V3 uses Multi-head Latent Attention (MLA) instead of standard GQA, compressing KV into a low-rank latent space.

1.3.11 Attention Pathologies

While the attention mechanism is powerful, it exhibits systematic failure modes that practitioners must understand—especially when scaling to long contexts or interpreting model behaviour.

Attention Sink

The phenomenon. Xiao et al. [393] discovered that transformer models allocate disproportionately high attention scores to the *first token* in the sequence—regardless of its semantic content. Even when the first token is a meaningless $\langle \text{BOS} \rangle$ marker, attention heads across all layers consistently attend to it, sometimes with 20–50% of total attention mass.

Why it happens. Softmax attention must produce a valid probability distribution ($\sum_j \alpha_j = 1$). When no key is particularly relevant to a query, the model needs a “dump” location for unused attention mass. During training, the first token becomes this default sink because it is always present and positionally predictable. It functions as a *no-op attention target*—the model has learned to route irrelevant attention there rather than distributing it unpredictably.

$$\alpha_{\text{sink}} = \frac{\exp(q^\top k_0 / \sqrt{d})}{\sum_j \exp(q^\top k_j / \sqrt{d})} \gg \frac{1}{n} \quad (\text{even when } k_0 \text{ is semantically irrelevant})$$

Consequences.

- **Streaming inference failure:** When using sliding-window KV caches, evicting the first token causes perplexity to spike catastrophically—the model loses its attention sink.
- **Misleading interpretability:** Naive attention visualizations suggest the first token is “important” when it is merely a mathematical artefact.
- **Context window waste:** The sink token occupies a KV cache slot without carrying useful information.

Solutions.

- **StreamingLLM [393]:** Always keep the first k tokens (“attention sinks”) in the KV cache alongside the recent sliding window. Enables infinite-length generation with bounded memory.
- **Sink tokens by design:** Some models (e.g., Mistral) prepend dedicated sink tokens during training that are explicitly meant to absorb residual attention.
- **Softmax alternatives:** Replace softmax with ReLU attention or sigmoid gating, where zero attention is representable without requiring a dump target.

Attention Dilution

The phenomenon. As sequence length n grows, each query must distribute its attention budget across more keys. The average attention weight per token decreases as $O(1/n)$, making it progressively harder for the model to concentrate on the few truly relevant positions—a problem known as *attention dilution* or *attention diffusion* [226].

The “Lost in the Middle” effect. Liu et al. [226] showed that LLMs exhibit a U-shaped retrieval curve: information placed at the *beginning* or *end* of long contexts is retrieved reliably, but information in the *middle* is often ignored. This is a direct consequence of attention dilution compounded with positional biases from RoPE/ALiBi:

Why it happens.

- **Softmax saturation:** With many keys, the softmax temperature effectively decreases, making the distribution more uniform (entropic).
- **Positional decay:** RoPE’s relative positional encoding introduces a natural decay with distance, suppressing attention to middle positions that are far from both start and end.
- **Training distribution:** Models trained on shorter sequences develop attention patterns biased toward recent context.

Mitigation strategies.

- **Explicit retrieval:** Place relevant context at the beginning or end of the prompt; use RAG to avoid relying on middle positions.
- **Long-context training:** Train on long documents with varied placement of key information [100].
- **Hierarchical attention:** Architectures like Mamba [123] or RWKV that avoid the $O(n^2)$ attention bottleneck entirely.
- **Landmark tokens:** Insert retrievable markers in the context that act as “signposts” for attention.
- **Temperature scaling:** Some implementations scale the attention logits by $\log n$ to counteract dilution in long sequences.

Other Attention Phenomena**Table 1.8:** Additional attention patterns observed in large transformers.

Pattern	Description	Implication
Attention heads specialization	Different heads learn distinct roles: syntax heads, co-reference heads, positional heads [361]	Not all heads are equally important; many can be pruned
Induction heads	Heads that implement $[A][B]\dots[A] \rightarrow [B]$ copying [273]	Critical for in-context learning; emerge in 2-layer+ models
Attention collapse	In deep networks, attention distributions can converge (all heads attend same positions)	Hurts expressivity; addressed by attention diversity losses
Retrieval heads	Specific heads specialize in retrieving factual information from context [387]	Explains why pruning certain heads causes hallucination spikes

1.3.12 Visualizing Attention for Explainability

Attention weights provide a window into model reasoning—but must be interpreted carefully.

Attention Visualization Methods

Raw attention maps. The simplest approach: plot the $n \times n$ attention matrix $A = \text{softmax}(QK^\top / \sqrt{d})$ as a heatmap for each head and layer. Tools like BertViz [359] render interactive multi-head visualizations.

Attention rollout. Raw attention at a single layer is misleading because information flows through residual connections across *all* layers. Abnar and Zuidema [2] propose *attention rollout*: multiply attention matrices across layers to approximate the total information flow from input to output:

$$R^{(l)} = A^{(l)} \cdot R^{(l-1)}, \quad R^{(0)} = I$$

where $A^{(l)}$ is the (averaged across heads) attention matrix at layer l , adjusted to include the residual connection: $A^{(l)} = 0.5 \cdot A_{\text{raw}}^{(l)} + 0.5 \cdot I$.

Gradient-weighted attention. Combine attention weights with gradient information to identify which attended tokens actually *influence* the output [20]:

$$\text{Relevance}(i) = \alpha_i \cdot \left| \frac{\partial y}{\partial h_i} \right|$$

This addresses the criticism that high attention $\neq$ high influence (a token can receive high attention but be processed through a near-zero-weight path).

Attention Is Not Explanation

Jain and Wallace [160] showed that attention weights often do not correlate with gradient-based feature importance and that adversarial attention distributions can produce identical outputs. Use attention visualization as a *hypothesis generator*, not as a faithful explanation. For causal attribution, prefer gradient-based methods, probing, or mechanistic interpretability.

Mechanistic Interpretability with Sparse Autoencoders (SAEs)

The interpretability problem. Individual neurons in transformer MLPs and residual streams are typically *polysemantic*—a single neuron activates for multiple unrelated concepts (e.g., “the colour blue AND academic citations AND the word ‘the’”). This makes direct neuron-level interpretation unreliable.

Sparse Autoencoders. Cunningham et al. [63] and Bricken et al. [30] demonstrated that training a sparse autoencoder (SAE) on model activations can decompose polysemantic representations into *monosemantic features*—interpretable directions that each correspond to a single concept:

$$h = W_{\text{dec}} \cdot \text{ReLU}(W_{\text{enc}} \cdot x + b_{\text{enc}}) + b_{\text{dec}}$$

where $W_{\text{enc}} \in \mathbb{R}^{m \times d}$ with $m \gg d$ (overcomplete basis), and the ReLU + sparsity penalty ensures only a few features activate per input.

Key findings from SAE interpretability:

- Features are *monosemantic*: each encodes a single human-interpretable concept (“code in Python,” “mentions of the Golden Gate Bridge,” “first-person narrative”) [30].
- Features are *steerable*: clamping a feature’s activation high/low directly controls model behaviour (e.g., forcing the “Golden Gate Bridge” feature on makes the model mention it in every response) [353].
- Features compose: complex behaviours emerge from combinations of simple features.
- SAEs scale: Templeton et al. [353] trained SAEs with up to 34M features on Claude 3 Sonnet, finding interpretable features for safety-relevant concepts (deception, sycophancy, dangerous requests).

SAE Training Recipe

1. Collect activations from a specific model layer across a large corpus.
2. Train a sparse autoencoder with L_1 penalty on the hidden layer: $\mathcal{L} = \|x - \hat{x}\|_2^2 + \lambda \|z\|_1$.
3. The learned encoder directions (W_{enc} rows) are candidate features.
4. Validate: for each feature, find max-activating examples and check semantic coherence.
5. Optionally: measure *feature absorption* and *dead features* to assess SAE quality.

Natural Language Autoencoders (Anthropic, 2026)

While SAEs decompose activations into interpretable *vectors*, their features still require human inspection of max-activating examples to understand. Anthropic’s Natural Language Autoencoders (NLAEs) [12] take a fundamentally different approach: they replace the sparse bottleneck with *natural language descriptions*, making interpretability automatic.

How NLAEs work.

1. **Encoder:** A language model reads the hidden activations (or the input text) and produces a natural language description of the active concepts: e.g., “The text discusses French cuisine and uses formal academic tone.”
2. **Decoder:** A second language model reads the natural language description and reconstructs the original activations (or predicts the next token).
3. **Training:** Both encoder and decoder are trained end-to-end to minimize reconstruction loss, with the bottleneck being a variable-length natural language string rather than a sparse vector.

Advantages over SAEs.

- **Self-interpreting:** Features are *literally* natural language—no manual labelling needed.
- **Compositional:** Can express complex, relational concepts (“a sarcastic response to a factual claim”) that SAE features cannot represent as single directions.
- **Hierarchical:** Descriptions can capture both fine-grained (word-level) and coarse (document-level) properties in the same representation.
- **Auditable:** The bottleneck description is human-readable, enabling direct inspection of what information the model “thinks” is present.

Limitations. NLAEs introduce a language-model-in-the-loop, making them computationally expensive and potentially subject to the same faithfulness concerns as any model-generated explanation. They also cannot easily represent sub-symbolic features (geometric patterns, exact numerical values) that SAEs handle naturally as activation magnitudes.

The Interpretability Stack

Think of interpretability tools as a hierarchy:

1. **Attention maps:** “What is the model looking at?” (cheapest, least faithful)
2. **Probing classifiers:** “What information is encoded at this layer?”
3. **Sparse Autoencoders:** “What monosemantic features are active?” (scalable, requires human labelling)
4. **Natural Language Autoencoders:** “What does the model think is happening?” (self-interpreting, expensive)

5. **Causal tracing / patching:** “Which components actually cause this output?” (most faithful, most expensive)

Each level trades off between cost, scalability, and faithfulness of explanation.

1.4 Prediction Heads: What Transformers Output

The transformer body produces contextual hidden states $\mathbf{h}_t \in \mathbb{R}^d$ for each position. What we *do* with these hidden states—the **prediction head**—defines the task. The same transformer backbone can serve radically different purposes simply by swapping the head.

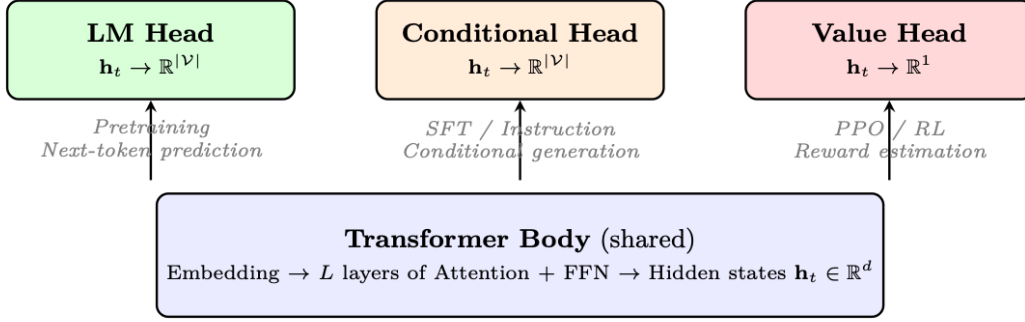


Figure 1.7: The same transformer backbone supports different tasks by swapping the prediction head. All three heads used in this paper share identical architecture below the final projection layer.

1.4.1 Language Modeling Head (Pretraining)

The standard LM head projects the final hidden state to vocabulary logits and trains with cross-entropy loss over the next token:

$$P(x_{t+1}|x_{\leq t}) = \text{softmax}(\mathbf{W}_{\text{head}} \cdot \mathbf{h}_t + \mathbf{b}) \quad (1.7)$$

where $\mathbf{W}_{\text{head}} \in \mathbb{R}^{|\mathcal{V}| \times d}$ (often tied with the embedding matrix: $\mathbf{W}_{\text{head}} = \mathbf{E}^T$).

LM Head Properties

- **Training objective:** Causal language modeling (predict next token for every position)
- **Loss:** $\mathcal{L}_{\text{LM}} = -\frac{1}{T} \sum_{t=1}^T \log P(x_t|x_{<t})$
- **Label:** Every token is both input (shifted right) and target (shifted left)
- **Used during:** Pretraining on large corpora (trillions of tokens)
- **Key insight:** The model learns general language understanding as a byproduct of next-token prediction

1.4.2 Conditional Generation Head (SFT / Instruction Following)

For supervised fine-tuning (SFT), the architecture is *identical* to the LM head—the same linear projection to vocabulary logits. The difference is purely in *what we compute loss on*:

$$\mathcal{L}_{\text{SFT}} = -\frac{1}{|y|} \sum_{t=1}^{|y|} \log P(y_t|x_{\text{prompt}}, y_{<t}) \quad (1.8)$$

Conditional Head – Key Differences from LM Head

- **Loss masking:** Only compute loss on the *response* tokens, not the prompt/instruction. The prompt provides context but no gradient signal.

- **Conditioning:** The model learns to generate responses *conditioned on* specific instruction formats (system prompts, user queries, tool calls).
- **Format tokens:** Special tokens (`<|user|>`, `<|assistant|>`) guide the model to produce structured outputs.
- **Used during:** SFT on curated instruction-response pairs; also during RL policy generation (the policy head that produces actions/responses).

Same Head – Different Training Signal

The LM head and SFT head are architecturally identical (same $\mathbf{W}_{\text{head}}$). The only difference is that during SFT, we mask the loss on prompt tokens. This subtle change transforms a general text predictor into a instruction-following assistant. The head learns to “activate” different generation modes based on the conditioning context.

1.4.3 Value Head (Regression for RL)

In reinforcement learning (PPO, GRPO), we need to estimate *how good* a state is—this requires a scalar output, not vocabulary logits. The **value head** replaces the LM projection with a simple regression layer:

$$V(s_t) = \mathbf{w}_{\text{value}}^T \cdot \mathbf{h}_t + b \in \mathbb{R} \quad (1.9)$$

where $\mathbf{w}_{\text{value}} \in \mathbb{R}^d$ and $b \in \mathbb{R}$.

Value Head Properties

- **Output:** Single scalar (expected cumulative reward from this state)
- **Loss:** MSE between predicted and actual returns: $\mathcal{L}_V = \frac{1}{T} \sum_t (V(s_t) - R_t)^2$
- **Architecture:** Linear layer $\mathbb{R}^d \rightarrow \mathbb{R}^1$ (sometimes with a small MLP: $d \rightarrow 256 \rightarrow 1$)
- **Backbone sharing:** Often shares the transformer body with the policy (with a separate value head), or uses a completely separate critic network
- **Used during:** PPO advantage estimation (GAE), reward model scoring

1.4.4 Head Selection Summary

Table 1.9: Prediction heads used throughout this paper and their training contexts.

Head	Output	Loss	Stage	Purpose
LM Head	$\mathbb{R}^{ \mathcal{V} }$	Cross-entropy (all tokens)	Pretraining	Learn language from raw text
Conditional Head	$\mathbb{R}^{ \mathcal{V} }$	Cross-entropy (response only)	SFT	Learn to follow instructions
Value Head	$\mathbb{R}^1$	MSE	RL (PPO)	Estimate state value for advantage
Reward Head	$\mathbb{R}^1$	Pairwise ranking	RM training	Score response quality

Head Initialization Matters

When adding a value head to a pretrained LM, initialize it near zero (small random weights). If initialized with large values, the initial value estimates will be wildly wrong, causing huge advantages and unstable PPO updates. Common practice: initialize the final linear layer with $\mathcal{N}(0, 1/\sqrt{d})$ or simply zeros.

1.4.5 HuggingFace Implementation

```

from transformers import (
    AutoModelForCausalLM,          # LM head (pretraining + SFT)
    AutoModelForSequenceClassification, # Reward head
    AutoTokenizer,
)
from trl import AutoModelForCausalLMWithValueHead # Value head (PPO)
import torch

model_name = "meta-llama/Llama-3.1-8B-Instruct"
tokenizer = AutoTokenizer.from_pretrained(model_name)

# === 1. LM Head (Pretraining / SFT) ===
# The default CausalLM model -- projects hidden states to vocab logits
lm_model = AutoModelForCausalLM.from_pretrained(
    model_name,
    torch_dtype=torch.bfloat16,
    device_map="auto",
)
# lm_model.lm_head: Linear(hidden_size -> vocab_size)
# Output: logits of shape (batch, seq_len, vocab_size)

inputs = tokenizer("The capital of France is", return_tensors="pt")
outputs = lm_model(**inputs)
next_token_logits = outputs.logits[:, -1, :] # (batch, vocab_size)
probs = torch.softmax(next_token_logits, dim=-1)

# === 2. Conditional Head (SFT) ===
# Architecturally identical to LM head -- difference is in loss masking
# During SFT, we only compute loss on response tokens:
messages = [
    {"role": "user", "content": "What is 2+2?"},
    {"role": "assistant", "content": "4"},
]
formatted = tokenizer.apply_chat_template(messages, return_tensors="pt")
labels = formatted.clone()
# Mask prompt tokens (set to -100 so cross-entropy ignores them)
prompt_len = len(tokenizer.apply_chat_template(messages[:1]))
labels[:, :prompt_len] = -100
loss = lm_model(input_ids=formatted, labels=labels).loss

# === 3. Value Head (PPO Critic) ===
# Adds a Linear(hidden_size -> 1) on top of the LM backbone
value_model = AutoModelForCausalLMWithValueHead.from_pretrained(
    model_name,
    torch_dtype=torch.bfloat16,
    device_map="auto",
)
# value_model.v_head: Linear(hidden_size -> 1)
# Returns both LM logits AND per-token value estimates

inputs = tokenizer("Explain quantum computing", return_tensors="pt")
lm_logits, loss, values = value_model(
    **inputs, return_dict=False
)
# values shape: (batch, seq_len, 1) -- scalar estimate per token

# === 4. Reward Head (Reward Model) ===
# Classification head: Linear(hidden_size -> 1) on last token
reward_model = AutoModelForSequenceClassification.from_pretrained(
    model_name,
    num_labels=1,          # single scalar output
    torch_dtype=torch.bfloat16,
    device_map="auto",
)

```

```
# Scores entire sequence by pooling the last token’s hidden state
inputs = tokenizer("Good response here", return_tensors="pt")
reward_score = reward_model(**inputs).logits # shape: (batch, 1)
```

Listing 1.3: Loading and using different prediction heads with HuggingFace.

Weight Tying: LM Head = Embedding Matrix Transposed

Most modern LLMs *tie* the LM head weights with the input embedding matrix: `lm_head.weight = model.embed_tokens.weight`. This means the LM head is *not* a separately learned layer—it reuses the embedding table. Benefits: fewer parameters ($|\mathcal{V}| \times d$ saved), better generalization, and the geometric structure of the embedding space directly determines token probabilities. You can verify this in HuggingFace: `model.lm_head.weight is model.model.embed_tokens.weight` returns `True` for most models.

1.5 Optimization Theory for LLM Training

Training a large language model means finding the set of parameters θ (billions of weights) that minimizes the loss function $\mathcal{L}(\theta)$ — typically the negative log-likelihood of the next token. This is an optimization problem in extraordinarily high-dimensional space, and the algorithm used to navigate this space determines whether training succeeds, diverges, or stalls.

1.5.1 Gradient Descent: The Foundation

What is a Gradient? The gradient $\nabla_{\theta}\mathcal{L}$ is a vector that points in the direction of *steepest increase* of the loss. Each component $\frac{\partial \mathcal{L}}{\partial \theta_i}$ tells us how much the loss would change if we slightly increased parameter θ_i . To *decrease* the loss, we move in the opposite direction:

$$\theta_{t+1} = \theta_t - \eta \nabla_{\theta} \mathcal{L}(\theta_t) \quad (1.10)$$

where $\eta > 0$ is the **learning rate** — the step size. This is **gradient descent** [310].

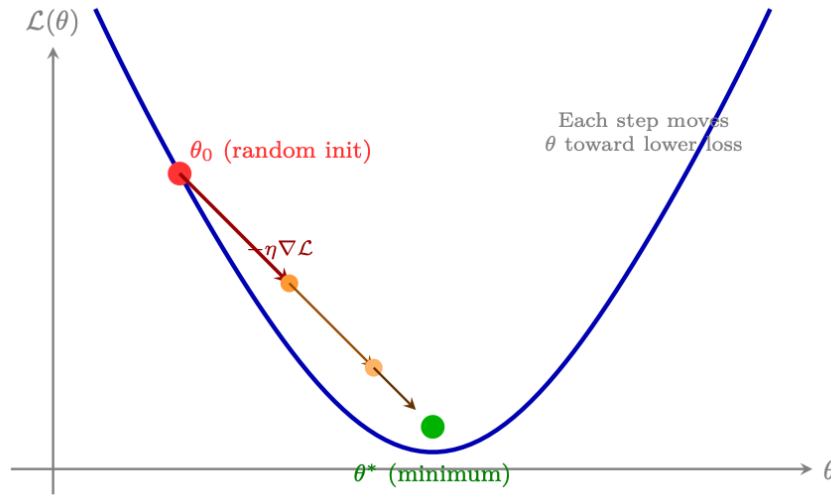


Figure 1.8: Gradient descent: starting from a random initialization θ_0 , each step moves the parameters in the direction that reduces the loss, with step size controlled by the learning rate η . The process converges toward a (local) minimum.

Why Full Gradient Descent is Impractical. Computing the exact gradient requires evaluating the loss over the *entire* training dataset (trillions of tokens for LLMs). This is computationally prohibitive — a single gradient step would require a full pass over all data.

Stochastic Gradient Descent (SGD). The solution: estimate the gradient from a small random subset (**mini-batch**) of the data [308]:

$$\nabla_{\theta} \mathcal{L}(\theta) \approx \frac{1}{B} \sum_{i=1}^B \nabla_{\theta} \ell(\theta; x_i)$$

where B is the batch size (typically 1K–4M tokens for LLMs). The mini-batch gradient is a *noisy but unbiased* estimate of the true gradient.

Why Mini-Batch SGD Works

- **Computational efficiency:** Each step costs $O(B)$ instead of $O(N_{\text{total}})$. With $B = 4096$ tokens and 15T total tokens, each step is ~ 4 billion $\times$ cheaper.
- **Noise as regularization:** The stochastic noise helps escape sharp local minima, finding flatter regions that generalize better.
- **GPU utilization:** Mini-batches are large enough to saturate GPU parallelism (matrix

multiplications become compute-bound rather than memory-bound).

- **Convergence:** Theoretically converges to a local minimum at rate $O(1/\sqrt{T})$ (slower than exact GD’s $O(1/T)$, but each step is millions of times cheaper).

From SGD to Adaptive Methods. While SGD with momentum works well for vision models (CNNs), LLM training requires **adaptive optimizers** — algorithms that maintain a per-parameter learning rate.

1.5.2 Why Vanilla SGD Fails for LLMs

Stochastic Gradient Descent updates weights as:

$$\theta_{t+1} = \theta_t - \eta \nabla_{\theta} \mathcal{L}(\theta_t)$$

SGD Problems for LLMs

- **Different gradient scales per layer:** Early layers in a transformer have much smaller gradients than later layers (vanishing gradients). A single learning rate η is too large for some parameters and too small for others.
- **Sparse gradients:** Embedding layers receive gradients only for tokens in the current batch. Most embedding rows have zero gradient. SGD with momentum wastes momentum on zero-gradient rows.
- **Saddle points:** High-dimensional loss landscapes have many saddle points. SGD can stall; adaptive methods escape faster.
- **Sensitivity to learning rate:** SGD requires careful tuning; a $2\times$ change in η can cause divergence.

1.5.3 Adam – Adaptive Moment Estimation

Adam [185] maintains per-parameter estimates of the first moment (mean of gradients) and second moment (uncentered variance of gradients).

Adam Update Equations

Given gradient $g_t = \nabla_{\theta} \mathcal{L}(\theta_t)$, hyperparameters $\beta_1, \beta_2, \epsilon, \eta$:

Step 1 – Update biased first moment estimate:

$$m_t = \beta_1 m_{t-1} + (1 - \beta_1) g_t$$

Step 2 – Update biased second moment estimate:

$$v_t = \beta_2 v_{t-1} + (1 - \beta_2) g_t^2$$

Step 3 – Bias correction:

$$\hat{m}_t = \frac{m_t}{1 - \beta_1^t}, \quad \hat{v}_t = \frac{v_t}{1 - \beta_2^t}$$

Step 4 – Parameter update:

$$\theta_{t+1} = \theta_t - \eta \cdot \frac{\hat{m}_t}{\sqrt{\hat{v}_t} + \epsilon}$$

Typical values: $\beta_1 = 0.9$, $\beta_2 = 0.95$ or 0.999 , $\epsilon = 10^{-8}$, $\eta = 10^{-4}$ to 10^{-5} .

What Each Term Does

- m_t (**momentum**): Exponential moving average of gradients. Smooths out noisy gradient estimates. $\beta_1 = 0.9$ means the current gradient contributes 10% and the history contributes 90%.
- v_t (**adaptive LR**): EMA of squared gradients. Parameters with consistently large gradients get a smaller effective learning rate ($\eta/\sqrt{v_t}$). Parameters with small gradients get a larger effective LR. This is the key to handling different gradient scales per layer.
- $\hat{m}_t, \hat{v}_t$ (**bias correction**): At $t = 1$, $m_1 = (1 - \beta_1)g_1$ is much smaller than the true mean. Dividing by $(1 - \beta_1^t)$ corrects this initialization bias. Without it, early steps are too small.
- ϵ (**numerical stability**): Prevents division by zero. Also acts as a floor on the effective learning rate.

1.5.4 AdamW – Decoupled Weight Decay

AdamW [238] fixes a subtle but important issue with how weight decay interacts with adaptive optimizers.

Why L2 Regularization $\neq$ Weight Decay in Adam

With L2 regularization, the loss becomes $\mathcal{L} + \frac{\lambda}{2}\|\theta\|^2$, so the gradient is $g_t + \lambda\theta_t$. In Adam, this regularization gradient is *scaled by the adaptive factor* $1/\sqrt{\hat{v}_t}$:

$$\theta_{t+1} = \theta_t - \eta \cdot \frac{\hat{m}_t + \lambda\theta_t}{\sqrt{\hat{v}_t} + \epsilon}$$

Parameters with large v_t (large gradient variance) get *less* regularization. This is not what we want – weight decay should be uniform.

AdamW – Decoupled Weight Decay

AdamW (Loshchilov & Hutter, 2017) applies weight decay *directly* to the parameters, outside the adaptive scaling:

$$\theta_{t+1} = \theta_t - \eta \cdot \frac{\hat{m}_t}{\sqrt{\hat{v}_t} + \epsilon} - \eta\lambda\theta_t$$

The weight decay term $\eta\lambda\theta_t$ is not divided by $\sqrt{\hat{v}_t}$. This gives uniform regularization across all parameters regardless of their gradient history.

Typical value: $\lambda = 0.1$ for LLM training.

Always Use AdamW – Never Plain Adam – for LLMs

The difference between Adam and AdamW is subtle but matters. With Adam + L2, the effective weight decay is stronger for parameters with small gradient variance (e.g., biases, LayerNorm parameters) and weaker for parameters with large gradient variance (e.g., attention weights). AdamW gives the intended uniform regularization. Most frameworks default to AdamW; double-check your optimizer class.

1.5.5 Muon: Beyond AdamW

For nearly a decade, AdamW was the unchallenged default optimizer for neural network training. Muon [224] (Liu et al., 2025) is the first serious challenger to that dominance. Its core insight is that the momentum buffer accumulated by Adam carries directional information that is obscured by the adaptive scaling step—and that *orthogonalizing* that momentum before applying it produces a fundamentally better update direction.

Muon: Orthogonalized Momentum

Muon replaces the standard Adam update for hidden-layer weight matrices with a two-step procedure:

1. **Orthogonalize the momentum:** Apply Newton-Schulz iteration to the momentum buffer M_t to produce an approximately orthogonal matrix $\tilde{M}_t = \text{NS}(M_t)$.
2. **Rescale to match AdamW RMS:** Scale $\tilde{M}_t$ so that its root-mean-square matches the RMS of the corresponding AdamW update, ensuring the effective step size is comparable.

The resulting update is applied to the weight matrix directly. Embedding layers, output projections, and scalar parameters continue to use AdamW.

The Newton-Schulz iteration is a matrix polynomial that converges to the orthogonal factor of the polar decomposition of M_t . Concretely, starting from $X_0 = M_t / \|M_t\|_F$, the iteration

$$X_{k+1} = \frac{3}{2}X_k - \frac{1}{2}X_k X_k^\top X_k$$

converges rapidly (typically 5–10 steps) to a matrix $\tilde{M}_t$ satisfying $\tilde{M}_t^\top \tilde{M}_t \approx I$. Orthogonality ensures that the update moves weight matrices in a direction that is maximally spread across all singular value directions—preventing the optimizer from collapsing updates onto a low-rank subspace, which is a known failure mode of Adam on weight matrices with highly skewed singular value spectra.

Empirical efficiency. Muon claims approximately **2× compute efficiency** relative to AdamW: the same validation loss is reached in roughly half the gradient steps. This is not a marginal improvement—it represents a qualitative shift in the optimizer’s sample efficiency on the loss landscape.

Adoption in frontier models. The momentum is clearly shifting toward Muon in large-scale pretraining:

- **GLM-4.5 / GLM-5** (Zhipu AI) [434, 435]: adopted Muon for pretraining
- **Kimi K2** (Moonshot AI) [17]: uses **MuonClip**, a variant that adds *QK-Clip*—rescaling query and key projections to cap attention logits and prevent loss spikes. Kimi K2 trained over 15.5T tokens with zero loss spikes, a stability record for models at that scale.
- **DeepSeek-V4** (2026) [71]: adopted Muon for its pretraining run

AdamW Is Still the Safe Default

AdamW remains the optimizer of choice for MAI-Thinking-1, Qwen, Llama, and most production fine-tuning pipelines. Muon’s advantages are most pronounced in large-scale pretraining of weight matrices; its behavior on LoRA adapters, small fine-tuning runs, and RL training (PPO/GRPO) is less well characterized. Use AdamW unless you are running a pretraining run at scale and have the infrastructure to validate Muon’s behavior on your specific architecture.

1.5.6 Learning Rate – The Most Important Hyperparameter

Typical Learning Rates by Training Phase

Phase	Typical LR	Notes
Pretraining (from scratch)	1e-4 to 3e-4	Large model, large batch
Continued pretraining	1e-5 to 1e-4	Smaller LR to preserve knowledge
SFT (supervised fine-tune)	1e-5 to 2e-5	Standard range
LoRA fine-tuning	1e-4 to 3e-4	Higher LR for adapter weights

For RL learning rates (PPO, DPO, GRPO) see §11.15.

1.5.7 Learning Rate Warmup

Why Warmup is Necessary

At the start of training, v_t (the second moment estimate) is initialized to zero. After bias correction: $\hat{v}_t = v_t / (1 - \beta_2^t)$. At $t = 1$ with $\beta_2 = 0.999$: $\hat{v}_1 = v_1 / (1 - 0.999) = 1000v_1$. This means the effective learning rate is $\eta / \sqrt{1000v_1}$ – much smaller than intended.

However, if the first gradient is unusually large (common at initialization), the second moment estimate can be dominated by this outlier, causing erratic early steps. Warmup mitigates this by starting with a very small LR and gradually increasing it, giving v_t time to accumulate a reliable estimate.

- **Linear warmup:** $\eta_t = \eta_{\max} \times t / T_{\text{warmup}}$
- **Typical warmup duration:** 1–5% of total steps for pretraining; 3–10% for fine-tuning (shorter runs need proportionally more warmup)
- **For SFT:** 50–200 warmup steps is typical

1.5.8 Learning Rate Schedules

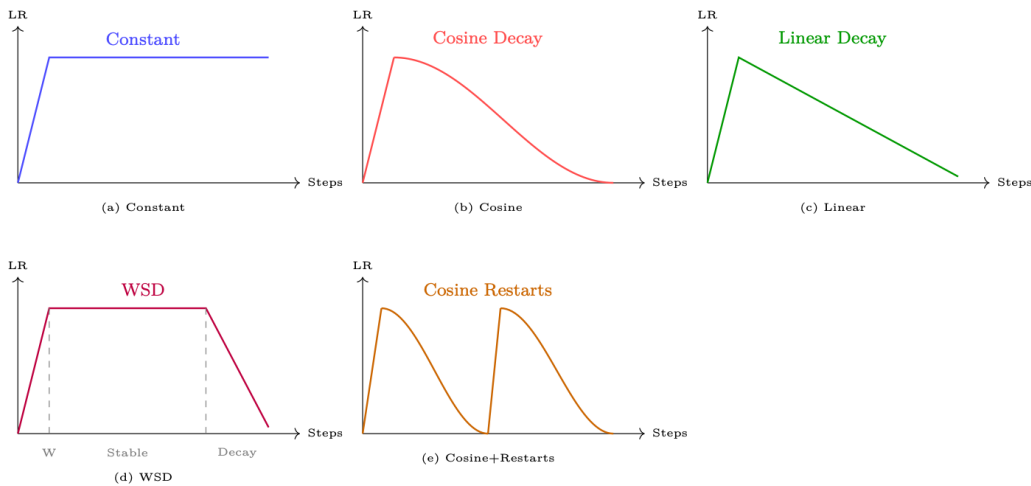


Figure 1.9: Common learning rate schedules. All include a linear warmup phase. WSD (Warmup-Stable-Decay) is the emerging standard for pretraining.

(a) Constant. Simplest schedule. Good for short fine-tuning runs where you want to avoid over-decaying the LR. Risk: no annealing means the model may not converge to the sharpest minimum.

(b) Cosine Decay.

$$\eta_t = \eta_{\min} + \frac{1}{2}(\eta_{\max} - \eta_{\min}) \left(1 + \cos \left(\frac{t - T_{\text{warmup}}}{T - T_{\text{warmup}}} \pi \right) \right)$$

Standard for pretraining and SFT. Smooth decay avoids abrupt LR changes. $\eta_{\min}$ is typically $\eta_{\max}/10$.

(c) Linear Decay. Simpler than cosine, similar empirical results. Preferred when you want predictable LR at any step.

(d) WSD – Warmup-Stable-Decay. The new standard for large-scale pretraining [118, 150]. Three phases:

1. **Warmup:** Linear ramp to $\eta_{\max}$ (1–5% of steps)
2. **Stable:** Constant $\eta_{\max}$ for the majority of training
3. **Decay:** Fast cosine or linear decay to $\eta_{\min}$ (last 10–20% of steps)

Key advantage: the stable phase allows checkpointing at any point and continuing training. The decay phase can be applied at the end of any run.

(e) Cosine with Restarts (SGDR). Periodic restarts reset the LR to $\eta_{\max}$. Can help escape local minima. Less common for LLMs; more useful for smaller models.

1.5.9 Gradient Clipping

Gradient Clipping

Gradient clipping rescales the gradient if its global norm exceeds a threshold:

$$g_t \leftarrow g_t \cdot \min \left(1, \frac{\tau}{\|g_t\|_2} \right)$$

where τ is `max_grad_norm` (typically 1.0).

Gradient Clipping vs. LR Reduction

Gradient clipping and reducing the learning rate both limit the size of parameter updates. The difference: clipping preserves the *direction* of the gradient (just scales the magnitude), while a smaller LR scales all updates uniformly. Clipping is better for handling occasional large gradients without slowing down normal training steps.

Putting It Together: HuggingFace Optimizer Configuration

The following snippet shows how the concepts from this section—AdamW with decoupled weight decay (§1.6.6), cosine learning-rate scheduling with linear warmup (§1.6.7), and gradient clipping (§1.6.8)—come together in practice using the HuggingFace `transformers` library.

```
from transformers import TrainingArguments, Trainer
from transformers import get_cosine_schedule_with_warmup
import torch

# --- Option 1: Using TrainingArguments (recommended) ---
training_args = TrainingArguments(
    output_dir="./checkpoints",

    # AdamW optimizer (decoupled weight decay, §1.6.6)
    optim="adamw_torch",
```

```

learning_rate=2e-5,          # peak LR after warmup
adam_beta1=0.9,              # first moment decay
adam_beta2=0.999,           # second moment decay
adam_epsilon=1e-8,           # numerical stability
weight_decay=0.01,           # decoupled L2 penalty

# Learning rate schedule (S1.6.7)
lr_scheduler_type="cosine",  # cosine decay to 0
warmup_ratio=0.1,           # 10% of steps = linear warmup

# Gradient clipping (S1.6.8)
max_grad_norm=1.0,          # clip by global L2 norm

# Mixed precision (S1.6.9)
bf16=True,                  # use BFloat16 on Ampere+ GPUs

# Training duration
num_train_epochs=3,
per_device_train_batch_size=8,
gradient_accumulation_steps=4, # effective batch = 8*4 = 32
)

trainer = Trainer(
    model=model,
    args=training_args,
    train_dataset=dataset,
)
trainer.train()

# --- Option 2: Manual control (for custom training loops) ---
from torch.optim import AdamW

# Separate weight-decay groups (don't regularize biases/norms)
no_decay = ["bias", "LayerNorm.weight", "layernorm.weight"]
param_groups = [
    {
        "params": [p for n, p in model.named_parameters()
                     if not any(nd in n for nd in no_decay)],
        "weight_decay": 0.01,
    },
    {
        "params": [p for n, p in model.named_parameters()
                     if any(nd in n for nd in no_decay)],
        "weight_decay": 0.0,
    },
]

optimizer = AdamW(param_groups, lr=2e-5, betas=(0.9, 0.999))

# Cosine schedule with linear warmup
total_steps = len(train_dataloader) * num_epochs
warmup_steps = int(0.1 * total_steps)
scheduler = get_cosine_schedule_with_warmup(
    optimizer,
    num_warmup_steps=warmup_steps,
    num_training_steps=total_steps,
)

# Training loop with gradient clipping
for batch in train_dataloader:
    outputs = model(**batch)
    loss = outputs.loss
    loss.backward()

    # Clip gradients before optimizer step
    torch.nn.utils.clip_grad_norm_(model.parameters(), max_norm=1.0)

```

```
optimizer.step()
scheduler.step()
optimizer.zero_grad()
```

Listing 1.4: Complete optimizer configuration combining AdamW – cosine schedule – and gradient clipping.

Practical Tips

- **Weight decay exclusion:** bias terms and layer-norm weights should not be regularized—they have few parameters and regularizing them hurts performance [238].
- **Warmup ratio:** 5–10% of total steps is standard; too little warmup with a high LR can destabilize early training.
- **Gradient accumulation:** simulates larger batches on limited GPU memory; clipping applies to the *accumulated* gradient.
- **BF16 vs. FP16:** prefer `bf16=True` on Ampere+ GPUs (wider dynamic range avoids loss scaling); fall back to `fp16=True` on older hardware.

1.5.10 Mixed Precision Training

BF16 vs. FP16

Format	Exponent bits	Mantissa bits	Dynamic range
FP32	8	23	$\sim 10^{-38}$ to 10^{38}
BF16	8	7	Same as FP32 (same exponent)
FP16	5	10	$\sim 6 \times 10^{-5}$ to 65504

BF16 Over FP16: Why Range Beats Precision in LLM Training

BF16 has the same exponent range as FP32, so it can represent the same range of values (just with less precision in the mantissa). FP16 has a much smaller dynamic range – gradients or activations that exceed 65504 cause overflow (NaN/Inf). This is why FP16 training requires *loss scaling* (multiplying the loss by a large constant to keep gradients in FP16 range), while BF16 training typically does not. A100 and H100 support BF16 natively; use BF16 unless you have a specific reason for FP16.

Loss Scaling (FP16 only).

1. Multiply loss by scale factor S (e.g., $S = 2^{15}$)
2. Compute gradients in FP16 (scaled by S)
3. Before optimizer step, divide gradients by S
4. Check for overflow (NaN/Inf); if found, skip step and reduce S
5. If no overflow for N consecutive steps, increase S

FP32 Master Weights. In mixed precision training, weights are stored in FP32 (master copy) and cast to BF16/FP16 for the forward/backward pass. The optimizer step is done in FP32. This is important because:

- Small gradient updates ($\Delta\theta \ll \theta$) would be lost in BF16 precision (7 mantissa bits $\approx 0.8\%$ relative precision)

- FP32 master weights ensure accurate accumulation of small updates over many steps
- Memory cost: $2\times$ weight storage (FP32 + BF16 copy)

When FP32 Master Weights Are Critical

FP32 master weights are most important for:

- Long training runs (many small gradient steps accumulate)
- Small learning rates (updates are tiny relative to weight magnitude)

For short SFT runs with large LR, BF16-only training (no FP32 master weights) often works fine and saves memory. For RL training, FP32 master weights are essential—see §11.15.

Mixed Precision in Practice: HuggingFace

```
# == HuggingFace TrainingArguments (simplest approach) ==
from transformers import TrainingArguments

# BF16 on Ampere+ GPUs (A100, H100, RTX 30xx/40xx)
args_bf16 = TrainingArguments(
    output_dir="./out",
    bf16=True,                # BF16 forward/backward; FP32 master weights
    bf16_full_eval=True,     # also use BF16 during evaluation
    # No loss scaling needed -- BF16 has FP32-equivalent range
)

# FP16 on older GPUs (V100, T4, RTX 20xx)
args_fp16 = TrainingArguments(
    output_dir="./out",
    fp16=True,               # FP16 forward/backward
    fp16_full_eval=False,    # keep eval in FP32 for accuracy
    # Loss scaling is automatic via PyTorch GradScaler
)

# == Manual PyTorch AMP (for custom training loops) ==
import torch

# Setup (PyTorch 2.x API)
use_fp16 = not torch.cuda.is_bf16_supported()
scaler = torch.amp.GradScaler("cuda", enabled=use_fp16) # only needed for FP16
optimizer = torch.optim.AdamW(model.parameters(), lr=2e-5)
dtype = torch.float16 if use_fp16 else torch.bfloat16

for batch in train_dataloader:
    optimizer.zero_grad()

    # Autocast: run forward pass in reduced precision
    with torch.autocast("cuda", dtype=dtype):
        outputs = model(**batch)
        loss = outputs.loss

    if use_fp16:
        # FP16 path: scale loss to prevent gradient underflow
        scaler.scale(loss).backward()
        scaler.unscale_(optimizer)          # unscale before clipping
        torch.nn.utils.clip_grad_norm_(model.parameters(), 1.0)
        scaler.step(optimizer)              # skips step on overflow
        scaler.update()                    # adjust scale factor
    else:
        # BF16 path: no scaling needed
        loss.backward()
        torch.nn.utils.clip_grad_norm_(model.parameters(), 1.0)
```

```
optimizer.step()

scheduler.step()
```

Listing 1.5: Mixed precision training with HuggingFace and manual PyTorch AMP.**Key Differences: BF16 vs. FP16 in Code**

- **BF16:** just wrap with `autocast(dtype=torch.bfloat16)`—no scaler needed. Simpler code and more numerically stable.
- **FP16:** requires `GradScaler` to prevent gradient underflow. The scaler dynamically adjusts a multiplier; if overflow is detected (NaN), the optimizer step is skipped and the scale is reduced.
- **Gradient clipping + FP16:** you *must* call `scaler.unscale_(optimizer)` before `clip_grad_norm_`, otherwise you’re clipping scaled gradients (wrong threshold).
- **Memory savings:** % reduction in activation memory (activations stored in 16-bit); weight memory depends on whether you keep FP32 master copies.

FP8 and FP4: Next-Generation Training Precision. DeepSeek-V3 [70] demonstrated that 671B-parameter training in FP8 (E4M3 for forward pass, E5M2 for backward) with fine-grained tile-wise scaling achieves less than 0.25% relative loss degradation versus BF16—at a total training cost of approximately \$5.6M. Key techniques include stochastic rounding to reduce quantization bias, per-tile scaling factors that adapt to local activation magnitudes, and keeping the first and last layers in higher precision where quantization sensitivity is greatest. NVIDIA’s Nemotron 3 pushes further to NVFP4 (4-bit), demonstrating stability over 25T training tokens by maintaining approximately 15% of layers in BF16. The economic implication is profound: careful precision engineering reduces frontier training costs by 3–5×, making billion-parameter training accessible beyond the largest labs.

1.5.11 Practical Optimizer Settings by Training Phase**Optimizer Hyperparameter Reference Table**

Phase	Optimizer	LR	WD	Warmup	Schedule
Pretraining	AdamW	3e-4	0.1	2000 steps	WSD or Cosine
SFT	AdamW	2e-5	0.01	100 steps	Cosine
LoRA SFT	AdamW	2e-4	0.01	100 steps	Cosine

All use: $\beta_1=0.9$, $\beta_2=0.95$, $\epsilon=10^{-8}$, `max_grad_norm=1.0`, BF16. For RL settings see §11.15.

Diagnosing Training Instability

```
# Monitor these metrics to diagnose optimizer issues:
# 1. Gradient norm -- should be < max_grad_norm most of the time
# 2. Loss scale (FP16) -- should be stable, not constantly decreasing
# 3. Parameter update norm -- should be << parameter norm

import torch

def log_optimizer_stats(model, optimizer, step):
    # Gradient norm (before clipping)
    total_norm = 0.0
    for p in model.parameters():
        if p.grad is not None:
            total_norm += p.grad.data.norm(2).item() ** 2
    total_norm = total_norm ** 0.5
```

```

# Adam second moment stats (proxy for adaptive LR)
v_norms = []
for group in optimizer.param_groups:
    for p in group['params']:
        state = optimizer.state[p]
        if 'exp_avg_sq' in state:
            v_norms.append(state['exp_avg_sq'].mean().item())

print(f"Step {step}: grad_norm={total_norm:.3f}, "
      f"mean_v={sum(v_norms)/len(v_norms):.6f}")

# Red flags:
# grad_norm >> 1.0 repeatedly -> reduce LR or increase warmup
# grad_norm == 0.0 -> gradient vanishing or wrong loss
# loss_scale decreasing -> FP16 overflow, switch to BF16
# v very small -> Adam not warmed up yet, extend warmup

```

The Learning Rate is the Most Important Hyperparameter

In practice, getting the learning rate right matters more than any other hyperparameter. A rule of thumb for LLM fine-tuning:

- Start with the values in the table above
- If loss diverges (increases after initial decrease): LR is too high
- If loss decreases very slowly and plateaus early: LR is too low
- If loss is unstable (oscillates): LR is too high or warmup is too short

The second most important hyperparameter is batch size (affects gradient noise and effective LR via the linear scaling rule). Everything else is secondary.

1.6 Flash Attention – Algorithm and Hardware Awareness

Flash Attention [64, 65] is one of the most impactful algorithmic innovations in deep learning since the transformer itself. It does not change the mathematical result of attention – it computes *exactly* the same output – but it restructures the memory access pattern so that the GPU's limited fast SRAM does all the heavy lifting, cutting HBM footprint from $O(n^2)$ to $O(n)$ and delivering 2–4× end-to-end wall-clock gains on typical workloads.

1.6.1 The Standard Attention Memory Problem

Standard scaled dot-product attention is:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right) V$$

Standard Attention Memory Complexity

For sequence length n and head dimension d :

- $Q, K, V \in \mathbb{R}^{n \times d}$: $O(nd)$ memory
- $S = QK^T \in \mathbb{R}^{n \times n}$: $O(n^2)$ **memory** – the bottleneck
- $P = \text{softmax}(S) \in \mathbb{R}^{n \times n}$: another $O(n^2)$
- $O = PV \in \mathbb{R}^{n \times d}$: $O(nd)$

At $n = 8192$, $d = 128$, BF16: the attention matrix alone is $8192^2 \times 2 \approx 134$ MB *per head*. With 32 heads, that is 4.3 GB just for one layer's attention scores.

Why $O(n^2)$ is Catastrophic

The attention matrix must be written to HBM (it does not fit in SRAM for long sequences), then read back for the softmax, then read again for the PV product. Each of these HBM round-trips is slow. For $n = 32768$ (32K context), the attention matrix is $32768^2 \times 2 \approx 2$ GB *per head* – completely infeasible to store.

1.6.2 The Flash Attention Key Insight – Tiling and Online Softmax

The core insight is: **we never need the full $n \times n$ matrix in memory at once**. We can compute the output O block-by-block if we use the *online softmax* trick.

Online Softmax. Recall that softmax requires a global maximum for numerical stability:

$$\text{softmax}(x_i) = \frac{e^{x_i - m}}{\sum_j e^{x_j - m}}, \quad m = \max_j x_j$$

The trick: we can *update* the running maximum and normalization factor as we process new blocks, without ever materializing the full row.

Online Softmax Update Rule

Given a running state $(m_{\text{old}}, \ell_{\text{old}}, O_{\text{old}})$ and a new block of scores s_{new} :

1. $m_{\text{new}} = \max(m_{\text{old}}, \max(s_{\text{new}}))$
2. $\ell_{\text{new}} = e^{m_{\text{old}} - m_{\text{new}}} \cdot \ell_{\text{old}} + \sum_j e^{s_{\text{new},j} - m_{\text{new}}}$
3. $O_{\text{new}} = \frac{1}{\ell_{\text{new}}} \left(e^{m_{\text{old}} - m_{\text{new}}} \cdot \ell_{\text{old}} \cdot O_{\text{old}} + e^{s_{\text{new}} - m_{\text{new}}} \cdot V_{\text{new}} \right)$

This is mathematically equivalent to computing softmax over all blocks at once.

1.6.3 The Flash Attention Algorithm

Flash Attention Forward Pass – Block Tiling

Setup: SRAM size M . Block sizes $B_r = \lceil M/(4d) \rceil$, $B_c = \min(\lceil M/(4d) \rceil, d)$.

1. Divide Q into $T_r = \lceil n/B_r \rceil$ blocks $Q_1, \dots, Q_{T_r}$
2. Divide K, V into $T_c = \lceil n/B_c \rceil$ blocks $K_1, \dots, K_{T_c}$
3. Initialize output $O \in \mathbb{R}^{n \times d}$, running max $m \in \mathbb{R}^n$, running sum $\ell \in \mathbb{R}^n$ (all in HBM)
4. **Outer loop** over $j = 1, \dots, T_c$:
 - (a) Load K_j, V_j from HBM to SRAM
 - (b) **Inner loop** over $i = 1, \dots, T_r$:
 - i. Load Q_i, O_i, m_i, ℓ_i from HBM to SRAM
 - ii. Compute $S_{ij} = Q_i K_j^T / \sqrt{d}$ (stays in SRAM)
 - iii. Apply online softmax update to get new m_i, ℓ_i, O_i
 - iv. Write O_i, m_i, ℓ_i back to HBM
5. Return O

Key: S_{ij} (the attention tile) is computed and discarded in SRAM. It is *never written to HBM*.

Flash Attention Complexity

	Standard Attention	Flash Attention
Memory (HBM)	$O(n^2)$	$O(n)$
HBM reads/writes	$O(n^2d)$	$O(n^2d/M)$
FLOPs	$O(n^2d)$	$O(n^2d)$ (same)
Speedup	$1\times$	$2\text{--}4\times$

In the forward pass, the total FLOPs remain $O(n^2d)$ – identical to standard attention. Flash Attention gains speed entirely by slashing slow HBM traffic, not by reducing arithmetic. (The backward pass actually performs *more* FLOPs due to recomputation, but the wall-clock time is still lower because the saved memory bandwidth dominates.)

1.6.4 Flash Attention 2 – Better Parallelism

Flash Attention 2 [64] made three key improvements:

1. **Reduced non-matmul FLOPs:** The original FA had unnecessary rescaling operations in the inner loop. FA2 restructures the loop to minimize these. On A100, Tensor Core matrix multiplications outpace scalar operations by roughly $16\times$, so even a small fraction of non-matmul work in the inner loop becomes the latency bottleneck.
2. **Better parallelism across sequence dimension:** FA1 parallelized over batch and heads only. FA2 also parallelizes over the query sequence dimension, enabling better GPU utilization for long sequences with small batch sizes.
3. **Causal masking optimization:** For autoregressive (causal) attention, roughly half the blocks are fully masked. FA2 skips these blocks entirely, giving $\sim 2\times$ speedup for causal attention vs. bidirectional.

1.6.5 Flash Attention 3 – Hopper Architecture

Flash Attention 3 [322] is designed specifically for H100 and exploits three Hopper-specific features:

- **TMA (Tensor Memory Accelerator):** H100 has a dedicated hardware unit for asynchronous bulk data movement between HBM and SRAM. FA3 uses TMA to overlap data loading with computation, hiding memory latency.
- **Warp-specialization:** FA3 assigns different warps to different roles (producer warps load data via TMA; consumer warps compute MMA). This is a software pipelining technique that keeps both the memory system and Tensor Cores busy simultaneously.
- **FP8 support:** H100 supports FP8 (E4M3/E5M2) Tensor Core operations at $2\times$ the throughput of BF16. FA3 supports FP8 attention with per-block quantization to maintain accuracy.

FA3 achieves up to **75% of H100 theoretical peak** for FP16 attention, compared to $\sim 35\%$ for FA2.

1.6.6 Flash Attention 4 – Blackwell Architecture

Flash Attention 4 [415] targets NVIDIA’s Blackwell GPUs (B200/GB200), which double Tensor Core throughput to 2.25 PFLOP/s (BF16) while non-matmul units (exponential, shared memory bandwidth) scale at a slower rate. This *asymmetric hardware scaling* means that the bottleneck shifts: on Blackwell, attention is limited not by matmul but by the softmax exponentials and shared memory traffic surrounding them.

FA4 addresses this with four key techniques:

- **Fully asynchronous MMA pipelines:** Blackwell’s MMA instructions are fully asynchronous (unlike Hopper’s wgmma which still blocked on completion). FA4 redesigns the pipeline to overlap MMA, TMA loads, and softmax rescaling across larger tile sizes, keeping all hardware units saturated.
- **Software-emulated exponential:** Instead of calling the hardware `ex2` unit (which is the throughput bottleneck), FA4 emulates e^x using polynomial approximations executed on the much faster Tensor Cores themselves. This trades extra matmul instructions for exponential-unit stalls.
- **Conditional softmax rescaling:** Standard FlashAttention rescales the running max every tile. FA4 skips the rescaling when the new tile’s max does not exceed the running max (common in practice), saving both register shuffles and synchronization barriers.
- **Tensor Memory + 2-CTA MMA mode (backward pass):** The backward pass uses Blackwell’s *Tensor Memory* (a per-SM scratchpad larger than shared memory) and a 2-CTA cooperative mode that fuses dQ accumulation across two thread-block clusters, halving shared memory round-trips.

FA4 Implementation: CuTe-DSL

FA4 is the first FlashAttention version written in **CuTe-DSL**, a Python-embedded domain-specific language for GPU kernels (part of CUTLASS 4.x). CuTe-DSL compiles 20–30× faster than C++ CUTLASS templates while retaining full control over register allocation and pipeline scheduling. This dramatically lowers the iteration time for kernel development.

Results. On B200 with BF16 head-dim 128 (causal, seq-len 8K):

- **1613 TFLOP/s** – 71% of Blackwell peak utilization
- **1.3×** faster than cuDNN 9.13 (NVIDIA’s proprietary fused kernel)
- **2.7×** faster than Triton on the same hardware

Hardware–Software Co-evolution

The FlashAttention series illustrates a key principle: each GPU generation shifts the bottleneck, demanding new algorithmic ideas rather than just re-compilation. A80 → memory bandwidth limited (FA1/FA2: tiling + recomputation). H100 → data movement limited (FA3: TMA + warp-specialization). B200 → non-matmul compute limited (FA4: software-emulated exp + conditional rescaling). Understanding *where the hardware bottleneck lies* is the prerequisite for writing efficient kernels.

1.7 Pretraining: Best Practices

Pretraining is the most expensive phase of LLM development—consuming millions of GPU-hours and requiring careful orchestration of data, compute, and hyperparameters. This section distills key lessons from Llama-3 [118], Chinchilla [140], and GPT-4 [274].

1.7.1 Training Objective

All modern decoder-only LLMs use **causal language modeling** (CLM):

$$\mathcal{L}_{\text{CLM}} = -\frac{1}{T} \sum_{t=1}^T \log P_{\theta}(x_t \mid x_{<t})$$

This simple objective—with enough data and scale—produces emergent capabilities (in-context learning, reasoning, instruction following) without explicit supervision [32].

1.7.2 Data Pipeline

Pretraining Data Recipe

- **Scale:** 1–15 trillion tokens for frontier models (Llama-3: 15T tokens)
- **Sources:** Web crawl (80%), code (10%), books/papers (5%), curated (5%)
- **Deduplication:** MinHash + exact substring dedup reduces memorization [203]
- **Quality filtering:** Perplexity-based classifier, heuristic filters (length, language ID, toxicity)
- **Data mixing:** Temperature-weighted sampling across domains; upweight code and math for reasoning

1.7.3 Scaling Laws

Hoffmann et al. [140] showed that compute-optimal training requires balancing model size N and data size D : $N_{\text{opt}} \propto C^{0.50}$, $D_{\text{opt}} \propto C^{0.50}$. A 70B model is compute-optimal at $\sim 1.4\text{T}$ tokens. In practice, models are *over-trained* (more tokens than Chinchilla-optimal) because inference cost scales with model size, not training tokens—smaller over-trained models are cheaper to deploy.

1.7.4 Key Hyperparameters**Table 1.10:** Pretraining hyperparameters from published models.

Setting	Llama-3 405B	Llama-3 8B	Qwen-2.5 72B	Mistral 7B
Tokens	15T	15T	18T	8T
Batch size (tokens)	16M	4M	4M	4M
Peak LR	8e-5	3e-4	3e-4	3e-4
Schedule	WSD	WSD	Cosine	Cosine
Weight decay	0.1	0.1	0.1	0.1
Context length	8192	8192	4096→32K	8192

1.7.5 Common Failure Modes**Pretraining Pitfalls**

- **Loss spikes:** Sudden loss increases from bad data batches or numerical instability. Llama-3 reports rolling back to checkpoints and skipping offending batches.
- **Memorization:** Model regurgitates training data verbatim. Fix: deduplicate aggressively; monitor extraction attacks.
- **Context length:** Training on short sequences then deploying at long context fails. Use continued pretraining on long documents + RoPE scaling.

1.7.6 Mid-Training: Preparing for Reinforcement Learning

A distinct pipeline stage has emerged between raw pretraining and post-training. *Mid-training* (sometimes called “continued pretraining” or “annealing”) up-weights STEM, mathematics, and code on high-quality curated data while extending context length to 128K–256K tokens. Its purpose is not to teach new knowledge but to *prepare the model for RL*: instilling the cognitive behaviors—verification, backtracking, subgoal decomposition—that reinforcement learning will later amplify.

Mid-Training Determines RL-Readiness

OctoThinker [236] demonstrated that mid-training composition determines whether a base model responds to RL at all. The same GRPO recipe produced strong reasoning improvements on Qwen (which has reasoning-dense mid-training) but failed entirely on Llama (which lacks it). The

difference was not model size or architecture—it was whether the base model already exhibited self-correction behaviors that RL could reinforce.

Typical mid-training uses 100B–500B tokens of curated high-quality data with a cosine-decayed learning rate. Llama 3’s “annealing tail,” MiMo’s 3-stage mixture (ramping math+code to 70%), and MAI-Thinking-1’s explicit STEM up-weight are all instances of this pattern.

1.8 Supervised Fine-Tuning (SFT)

SFT transforms a pretrained language model into an instruction-following assistant by training on curated prompt–response pairs. This is the bridge between raw language modeling and RLHF.

1.8.1 SFT Objective

The loss is identical to CLM, but computed only on **response tokens**:

$$\mathcal{L}_{\text{SFT}} = -\frac{1}{|y|} \sum_{t=1}^{|y|} \log P_{\theta}(y_t \mid x_{\text{prompt}}, y_{<t})$$

Prompt tokens provide context but receive no gradient (labels set to -100).

1.8.2 Data Quality: The LIMA Principle

Zhou et al. [439] demonstrated that 1,000 carefully curated examples can match models trained on 50K+ noisy examples. Key requirements:

- **Diversity:** Cover QA, summarization, code, math, creative writing, multi-turn dialogue
- **Correctness:** Every response must be factually accurate and well-formatted
- **Length balance:** Mix short (1-sentence) and long (multi-paragraph) responses
- **Decontamination:** Remove overlap with evaluation benchmarks

1.8.3 Training Configuration

```
from trl import SFTTrainer, SFTConfig

sft_config = SFTConfig(
    output_dir="./sft_output",
    max_seq_length=4096,
    packing=True,                # Pack short examples into full sequences
    learning_rate=2e-5,
    lr_scheduler_type="cosine",
    warmup_ratio=0.1,
    weight_decay=0.01,
    max_grad_norm=1.0,
    num_train_epochs=3,
    per_device_train_batch_size=4,
    gradient_accumulation_steps=8,
    bf16=True,
    gradient_checkpointing=True,
)
trainer = SFTTrainer(model=model, args=sft_config,
                    train_dataset=dataset, processing_class=tokenizer)
trainer.train()
```

1.8.4 Efficient Training Solutions

Standard HuggingFace training leaves significant performance on the table. Several libraries provide drop-in efficiency gains for SFT workloads:

Liger Kernel [145]. An open-source set of **Triton-fused kernels** from LinkedIn that replace standard PyTorch operators during training. Key fusions include:

- **Fused Cross-Entropy:** Merges the final linear projection, softmax, and loss computation into a single kernel—avoids materializing the full (batch $\times$ seq $\times$ vocab) logit tensor.
- **Fused RMSNorm / SwiGLU / RoPE:** Eliminates intermediate memory allocations for common LLM building blocks.
- **Chunked operations:** Processes large tensors in tiles to keep peak memory bounded.

Result: 20% higher throughput and up to 60% memory reduction with a one-line integration (`apply_liger_kernel_to_llama()`). Compatible with FSDP, DeepSpeed, and LoRA.

Unsloth [132]. A specialized fine-tuning library that combines **custom CUDA/Triton kernels** with aggressive memory optimization:

- Manual backpropagation through LoRA layers (avoids autograd overhead).
- 4-bit QLoRA with fused dequantization—trains 70B models on a single 48 GB GPU.
- Intelligent RoPE and attention kernel fusion specific to each architecture (Llama, Mistral, Qwen, Gemma).

Result: 2–5 $\times$ faster than vanilla HuggingFace + PEFT, with 60–70% less VRAM. Particularly impactful for single-GPU and consumer-hardware workflows.

torch tune [349]. Meta’s native PyTorch fine-tuning library (development wound down in 2025), designed around **composability** rather than monolithic abstractions:

- Pure PyTorch—no trainer class; recipes are readable single-file scripts.
- Native integration with `torch.compile`, FSDP2, and activation checkpointing.
- First-class support for QLoRA, full fine-tuning, and knowledge distillation.
- Built-in quantization-aware training (QAT) for post-training compression.

Result: Comparable speed to custom solutions but with full debuggability and no framework lock-in.

Choosing an Efficiency Stack

- **Quick LoRA/QLoRA on ≤ 1 GPU:** Unsloth (fastest time-to-train, minimal setup)
- **Multi-GPU full fine-tune:** TRL/DeepSpeed + Liger Kernel (best throughput at scale)
- **Research / custom training loops:** torchtune (transparent, hackable, native PyTorch)

These are *complementary*: Liger kernels can be used inside both TRL and torchtune workflows.

1.8.5 Best Practices

Table 1.11: SFT training guidelines.

Practice	Details
Packing	Concatenate multiple short examples into one sequence (separated by EOS). Avoids padding waste.
NEFTune [159]	Add uniform noise to embeddings ($\alpha = 5$). Improves MT-Bench by 5–15% at zero cost.
Chat template	Always use the model’s native template. Mismatched templates degrade quality.
Epochs	2–3 for large datasets; up to 5 for small (<10K) curated sets. Over-training causes format memorization.

SFT Is Not Enough

SFT teaches format and basic instruction following, but cannot reliably teach: *preference* (which response is better—needs RLHF/DPO), *refusal* (when not to answer—needs safety training), *calibration* (saying “I don’t know”—needs RL with truthfulness rewards), or *complex reasoning* (multi-step chains—needs RL with verifiable rewards). The full pipeline is: Pretrain $\rightarrow$ SFT $\rightarrow$ RLHF/DPO.

1.9 LoRA and Parameter-Efficient Fine-Tuning

Full fine-tuning of a 70B model requires storing 70B trainable parameters plus their optimizer states (560+ GB of memory). LoRA [146] (Low-Rank Adaptation) provides a way to fine-tune with <1% of the parameters while achieving comparable quality.

1.9.1 The LoRA Insight

LoRA Core Idea

Instead of updating a full weight matrix $W \in \mathbb{R}^{d \times d}$, learn a low-rank perturbation:

$$W' = W + \frac{\alpha}{r} \cdot BA, \quad B \in \mathbb{R}^{d \times r}, \quad A \in \mathbb{R}^{r \times d}$$

- W is **frozen** (no gradients, no optimizer states)
- Only B and A are trained: $2 \times d \times r$ parameters instead of d^2
- At rank $r = 16$, $d = 4096$: LoRA adds $2 \times 4096 \times 16 = 131K$ params per layer vs. $16.8M$ for full matrix
- α/r scaling controls the magnitude of the update

Why Low-Rank Works

Aghajanyan et al. [3] showed that fine-tuning operates in a very low-dimensional subspace — the “intrinsic dimensionality” of the fine-tuning task is much smaller than the model’s parameter count. A 175B model’s fine-tuning task may have intrinsic dimensionality <10,000. LoRA exploits this directly: rank r constrains the update to an r -dimensional subspace per weight matrix.

Why the α/r Scaling Matters

Without scaling, doubling the rank r would roughly double the magnitude of $\Delta W = BA$ (more columns in B contribute to the sum). This means changing rank would also change how much the model is perturbed—you’d need to re-tune the learning rate every time you adjust r .

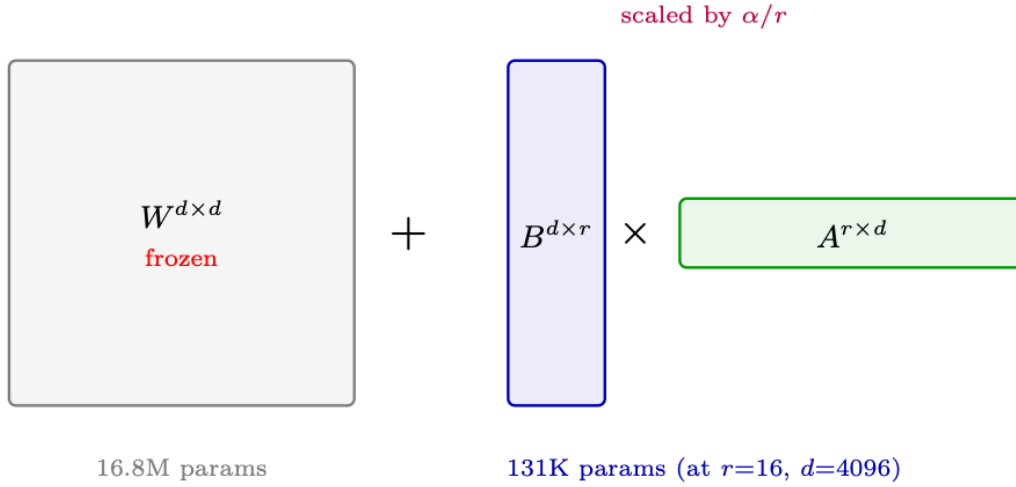
LoRA: Low-Rank Decomposition of Weight Update

Figure 1.10: LoRA decomposes the weight update ΔW into two small matrices $B \times A$. The original weight W remains frozen; only B and A receive gradients. At inference, the product BA can be merged into W with zero overhead.

The α/r factor **normalizes the update magnitude** so that it stays approximately constant regardless of rank:

$$W' = W + \frac{\alpha}{r} \cdot BA$$

- **Fix α , sweep r :** The effective update magnitude stays $\sim \alpha$ regardless of rank. You can try $r \in \{8, 16, 32, 64\}$ without re-tuning LR.
- **Common practice:** Set $\alpha = r$ (so $\alpha/r = 1$) or $\alpha = 2r$ (so $\alpha/r = 2$). This is a convenient default where the scaling factor is a small integer.
- **Why not just tune LR?** You could, but α/r provides a *rank-independent* knob. Teams can share LR recipes across experiments with different ranks.
- **rsLoRA insight [176]:** At high ranks ($r \geq 64$), empirical evidence shows $\alpha/\sqrt{r}$ is more stable than α/r , because the variance of BA scales with $\sqrt{r}$, not r .

1.9.2 LoRA Hyperparameters

Choosing LoRA hyperparameters correctly is critical — the wrong rank or alpha can either under-fit (too constrained) or waste memory (too expressive).

Rank Selection Rules of Thumb

- **r=8:** Simple tasks (single-domain chat, classification). Very memory-efficient.
- **r=16:** General-purpose fine-tuning. Good default.
- **r=32–64:** Complex tasks (math, code, multi-turn reasoning). Approaches full fine-tune quality.
- **r=128+:** Diminishing returns; consider full fine-tuning or QLoRA with higher rank.
- **Key indicator:** If training loss plateaus well above full fine-tune loss, increase rank.

Table 1.12: LoRA hyperparameter guide.

Hyperparameter	Typical Values	Guidance
r (rank)	8, 16, 32, 64	Higher = more capacity but more memory. Start with 16.
lora_alpha	16, 32 (often = r or $2r$)	Controls update magnitude via α/r scaling.
target_modules	q_proj, k_proj, v_proj, o_proj	All attention projections. Add gate_proj, up_proj, down_proj for full coverage.
lora_dropout	0.0–0.1	Regularization. Usually 0.05 for small datasets.
bias	"none"	Training biases adds minimal params but rarely helps.
Learning rate	1e-4 to 3e-4	Higher than full fine-tuning (only adapters update).

Table 1.13: LoRA variants and their innovations.

Method	Key Innovation	When to Use
QLoRA [75]	4-bit quantized base + LoRA in BF16. NF4 data type + double quantization.	Fine-tune 70B on single 48GB GPU.
DoRA [228]	Decomposes W into magnitude and direction; LoRA updates direction only.	Better generalization for reasoning.
LoRA+ [136]	Different LRs for A/B ($\eta_B = \lambda\eta_A$, $\lambda \approx 16$).	Free 2% gain; no extra cost.
AdaLoRA [424]	Dynamic rank budget across layers (SVD-based importance).	Very tight compute budget.
rsLoRA [176]	Scales by $\alpha/\sqrt{r}$ instead of α/r . Stable at high ranks.	When using $r \geq 64$.
VeRA [190]	Shared frozen random A, B ; trains diagonal scaling only.	Extreme param efficiency.
LoRA-FA	Freezes A after init; only trains B . Halves LoRA memory.	Memory-constrained scenarios.

1.9.3 LoRA Variants

Key Extensions Explained

DoRA – Weight-Decomposed Low-Rank Adaptation. DoRA [228] observes that full fine-tuning tends to change the *direction* of weight vectors more than their magnitude. Standard LoRA conflates both. DoRA decomposes each weight column into magnitude $m = \|W\|_{\text{col}}$ and direction $\hat{V} = W/\|W\|_{\text{col}}$, then applies LoRA only to the direction:

$$W' = m \odot \hat{V}', \quad \hat{V}' = \frac{W + BA}{\|W + BA\|_{\text{col}}}$$

Magnitude m is a separate learnable vector (one scalar per column). This consistently outperforms LoRA by 1–3% on reasoning and instruction-following benchmarks with no additional inference cost (merged at deployment).

LoRA+ – Asymmetric Learning Rates. Hayou et al. [136] show that matrices A and B in LoRA have different optimal learning rates. Since B is initialized to zero, it starts in a very different regime than A (initialized from $\mathcal{N}(0, \sigma^2)$). Setting $\eta_B \approx 16 \times \eta_A$ improves convergence speed and final quality by $\sim 2\%$ — a free gain requiring only a one-line config change:

```
# LoRA+ in PEFT: set different LRs per matrix
optimizer_grouped_parameters = [
    {"params": [p for n, p in model.named_parameters() if "lora_B" in n],
     "lr": 2e-4 * 16},      # B matrix: higher LR
    {"params": [p for n, p in model.named_parameters() if "lora_A" in n],
     "lr": 2e-4},           # A matrix: base LR
]
```

VeRA – Vector-based Random Matrix Adaptation. VeRA [190] takes parameter efficiency to the extreme: instead of learning A and B , it *freezes* them as shared random matrices across all layers and only trains two diagonal scaling vectors $d_b \in \mathbb{R}^r$ and $d_a \in \mathbb{R}^d$:

$$\Delta W = B \cdot \text{diag}(d_b) \cdot A \cdot \text{diag}(d_a)$$

This reduces trainable parameters by $\sim 10\times$ vs. LoRA (only $r + d$ params per layer) while achieving 90–95% of LoRA quality. Best for scenarios where you need hundreds of task-specific adapters with minimal storage.

QLoRA Memory Savings

70B model full fine-tune: 140 GB (weights) + 280 GB (optimizer) + 140 GB (gradients) = 560 GB ($7 \times$ A100-80GB).

70B QLoRA (r=16, all linear layers):

- Base model in NF4: $70\text{B} \times 0.5 = 35$ GB
- LoRA adapters in BF16: ~ 160 MB
- Optimizer states (only for adapters): ~ 320 MB
- Activations (gradient checkpointing): ~ 8 GB
- **Total: ~ 44 GB** — fits in a single 48GB GPU!

```
# QLoRA configuration with PEFT
from peft import LoraConfig, get_peft_model, prepare_model_for_kbit_training
from transformers import BitsAndBytesConfig
import torch

# 4-bit quantization config
bnb_config = BitsAndBytesConfig(
    load_in_4bit=True,
    bnb_4bit_quant_type="nf4",           # NormalFloat4 - optimal for weights
    bnb_4bit_compute_dtype=torch.bfloat16, # Compute in BF16
    bnb_4bit_use_double_quant=True,      # Quantize the quantization constants
)

# LoRA config
lora_config = LoraConfig(
    r=16,
    lora_alpha=32,                       # alpha/r = 2x scaling
    target_modules=["q_proj", "k_proj", "v_proj", "o_proj",
                   "gate_proj", "up_proj", "down_proj"],
    lora_dropout=0.05,
    bias="none",
    task_type="CAUSAL_LM",
)
```

```

model = prepare_model_for_kbit_training(model) # Prepare for QLoRA
model = get_peft_model(model, lora_config)      # Add LoRA adapters
model.print_trainable_parameters()
# Output: trainable params: 83,886,080 || all params: 70,553,706,496 || 0.12%

```

1.9.4 Other PEFT Approaches

LoRA dominates modern practice, but it is not the only parameter-efficient method. For completeness, the main alternatives:

Table 1.14: PEFT method families. LoRA is the de facto standard for LLM fine-tuning; the others are included for historical context and niche use cases.

Method	Mechanism	Pros / Cons	Status
LoRA [146] (and variants)	Low-rank matrices added to existing weights	Mergeable at inference (zero overhead); well-supported; works for all architectures	Standard
Adapters [144]	Small bottleneck MLPs inserted between layers	Modular; stackable; adds inference latency (extra sequential layers)	Rarely used
Prefix Tuning [211]	Learnable “virtual tokens” prepended to keys/values at each layer	No weight modification; effective for generation tasks; consumes context length	Niche
Prompt Tuning [204]	Learnable soft prompt embeddings prepended to input	Extremely few params (<0.01%); weaker than LoRA for complex tasks	Niche
IA3 [222]	Learned vectors that rescale keys, values, and FFN activations	Even fewer params than LoRA; mergeable; limited capacity	Deprecated
BitFit [417]	Train only bias terms	Near-zero params; surprisingly effective for simple tasks; limited expressiveness	Historical

Why LoRA Won

LoRA became the standard because it uniquely combines: (1) **zero inference overhead** — adapters merge into base weights, unlike Adapters or Prefix Tuning which add latency or consume context; (2) **composability** — multiple LoRA adapters can be swapped at serving time for multi-tenant deployments; (3) **ecosystem support** — HuggingFace PEFT, TRL, vLLM, and all major frameworks have first-class LoRA support; (4) **proven at scale** — used in production by Meta, Google, and most open-source fine-tunes on HuggingFace. Unless you have a specific constraint that LoRA cannot satisfy, it should be your default choice.

1.10 Mixture of Experts (MoE)

Mixture of Experts models [166, 325] scale model capacity without proportionally scaling compute cost by activating only a subset of parameters for each token.

1.10.1 Architecture

MoE Layer

In a MoE transformer, the FFN layer in each block is replaced by N parallel “expert” FFNs plus a **router** that selects which experts to use:

$$\text{MoE}(x) = \sum_{i=1}^N g_i(x) \cdot E_i(x), \quad g(x) = \text{TopK}(\text{softmax}(W_r x))$$

- E_i are expert networks (standard FFN layers)
- $g_i(x)$ are gating weights from the router (only top- K are non-zero)
- Typically $K = 2$ out of $N = 8\text{--}64$ experts are active per token
- Total params scale with N ; **active params** scale with K/N of FFN size

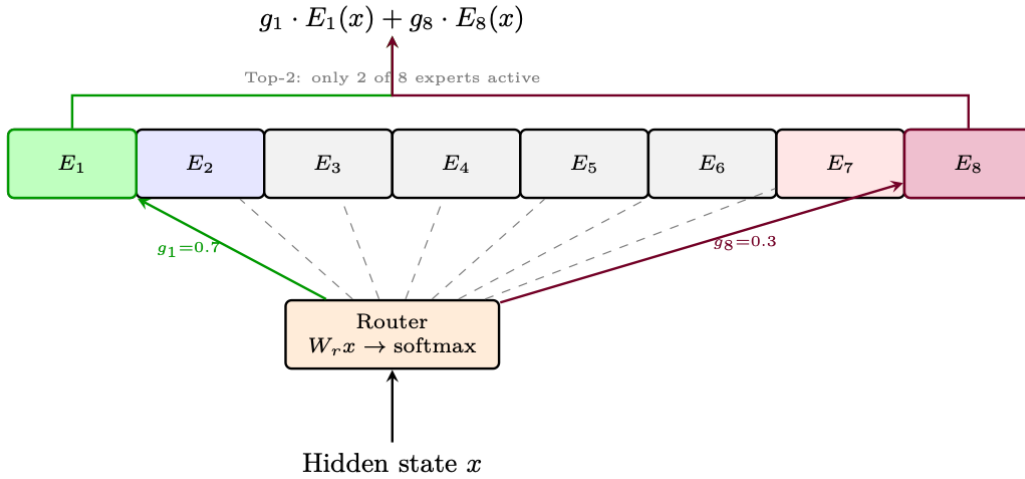


Figure 1.11: MoE layer with 8 experts and Top-2 routing. Only the two highest-gated experts are computed per token; the rest are skipped entirely.

1.10.2 Load Balancing

The Load Balancing Problem

Without constraints, the router may send most tokens to the same 1–2 experts (“expert collapse”). This wastes capacity and creates compute imbalance across GPUs (each expert typically lives on a different GPU).

Solution: Add an auxiliary load-balancing loss:

$$\mathcal{L}_{\text{bal}} = \alpha \cdot N \sum_{i=1}^N f_i \cdot p_i$$

where f_i = fraction of tokens routed to expert i , p_i = mean router probability for expert i . This encourages uniform expert utilization.

1.10.3 Noisy Top-K Gating: Making Discrete Routing Trainable

The core challenge in MoE is that **top- k selection is not differentiable** — you can’t backpropagate through a hard “pick the top 2” operation. The field has developed two key tricks to solve this:

The Routing Differentiability Problem

The router computes logits $h(x) = W_r \cdot x$ for each expert, then selects the top- k . But:

- The *selected* experts get gradients through their gate weights (softmax over selected)
- The *selection decision itself* (which k to pick) has zero gradient
- Without a trick, the router can get stuck: an expert never selected $\rightarrow$ never gets a gradient signal $\rightarrow$ never gets selected

Approach 1: Noisy Top-K Gating [325]. Add learnable Gaussian noise to the router logits *before* the top- k selection:

$$\begin{aligned}
 h(x) &= W_g \cdot x && \text{(clean logits)} \\
 H(x) &= h(x) + \epsilon \cdot \text{Softplus}(W_{\text{noise}} \cdot x), \quad \epsilon \sim \mathcal{N}(0, 1) && \text{(noisy logits)} \\
 \text{TopK}(v, k)_i &= \begin{cases} v_i & \text{if } v_i \text{ is in the top } k \\ -\infty & \text{otherwise} \end{cases} && (1.11) \\
 g(x) &= \text{softmax}(\text{TopK}(H(x), k)) && \text{(sparse gates)}
 \end{aligned}$$

- W_{noise} is a *learned* noise magnitude — the model learns how much exploration each expert needs
- During training, noise occasionally promotes “underdog” experts into the top- k , giving them gradient signal
- At inference, noise is removed: use clean logits $h(x)$ for deterministic routing
- The Softplus ensures noise scale is always positive

Approach 2: Gumbel-Softmax Trick (for differentiable discrete sampling). An alternative from the variational inference literature [161]. The **Gumbel-Max trick** provides exact sampling from a categorical distribution:

$$z = \arg \max_i [\log \pi_i + G_i], \quad G_i \sim \text{Gumbel}(0, 1) \quad (1.12)$$

where Gumbel noise is generated as $G_i = -\log(-\log(U_i))$, $U_i \sim \text{Uniform}(0, 1)$.

For **top- k routing**: taking the top- k of $(\log \pi_i + G_i)$ gives k samples *without replacement* from the categorical distribution defined by π .

Since $\arg \max$ is non-differentiable, the **Gumbel-Softmax** relaxation replaces it with a temperature-controlled softmax:

$$\hat{g}_i = \frac{\exp((\log \pi_i + G_i)/\tau)}{\sum_j \exp((\log \pi_j + G_j)/\tau)} \quad (1.13)$$

- $\tau \rightarrow 0$: approaches a hard one-hot (exact but non-differentiable)
- $\tau \rightarrow \infty$: approaches uniform (differentiable but uninformative)
- In practice, anneal τ from 1.0 down to 0.1–0.5 during training
- **Straight-through estimator**: use hard top- k in the forward pass, but Gumbel-Softmax gradients in the backward pass — best of both worlds

Which Approach Is Used in Practice?

- **Sparsely-Gated MoE** [325], **Mixtral** [166], **DeepSeek-V2** [69]: Use Noisy Top-K with Gaussian noise. Simple, effective, well-proven at scale.
- **Switch Transformer** [89]: Simplified to Top-1 with no noise (relies on load-balancing loss alone).
- **Research / smaller-scale MoE**: Some use Gumbel-Softmax for fully differentiable routing, especially when learning the routing itself is the research objective.
- **Key insight**: Both approaches solve the same problem (making discrete selection trainable) via noise injection. Gaussian noise is simpler; Gumbel noise has stronger theoretical guarantees for categorical sampling.

Auxiliary-Loss-Free Load Balancing. The standard approach adds a balance penalty to the training loss, but DeepSeek [372] demonstrated that this fights the routing signal: the loss pushes toward uniform utilization while the router tries to specialize. Their solution moves balancing entirely out of the gradient into a per-expert routing *bias* updated based on observed utilization—achieving both better quality *and* more expert specialization. A subsequent insight from Qwen revealed that the *aggregation scope* matters more than the mechanism: computing balance statistics per micro-batch (the common default) silently destroys specialization because micro-batches are too homogeneous. Aggregating over the full global batch—as MAI-Thinking-1 confirms—restores the diversity signal that experts need to differentiate.

1.10.4 Notable MoE Models

Model	Total Params	Active Params	Experts	Innovation
Switch Transformer [89]	1.6T	100B	128, Top-1	First large-scale MoE; simplified routing
Mixtral 8x7B [166]	47B	13B	8, Top-2	Open-weight; matches Llama-2 70B quality
DeepSeek-V2 [69]	236B	21B	160, Top-6	DeepSeekMoE with shared + routed experts
Qwen-MoE [350]	14.3B	2.7B	60, Top-4	Fine-grained experts for efficiency
DBRX [67]	132B	36B	16, Top-4	Fine-grained with 4 experts per block

1.11 Diversity in LLM Training

Diversity — in training data, model outputs, and optimization trajectories — is critical for preventing mode collapse and ensuring robust, general-purpose LLMs. This section covers the key diversity mechanisms applicable to all LLM training phases.

1.11.1 Sampling Diversity

Sampling Strategies for Diverse Generation

- **Temperature τ :** $P(x_i) \propto \exp(\text{logit}_i/\tau)$. Higher τ = more uniform distribution = more diverse. Typical: $\tau = 0.7$ – 1.0 for RLHF generation.
- **Top- k :** Only sample from the k highest-probability tokens. Prevents degenerate low-probability tokens.
- **Top- p (nucleus):** Sample from the smallest set of tokens whose cumulative probability $\geq p$. Adaptive: more diverse when the model is uncertain.
- **Min- p :** Only keep tokens with $P \geq p_{\min} \times P_{\max}$. More principled than top- k .
- **Frequency/presence penalty:** Penalize tokens that appeared in the response. Encourages lexical diversity.

1.11.2 Training Data Diversity

- **Prompt diversity:** Cover different domains, difficulty levels, and formats. The Goldilocks principle: prompts should have 20–80% success rate.
- **Deduplication:** Remove near-duplicate training examples (MinHash, n-gram overlap). Duplicates cause overfitting to specific patterns.
- **Data mixing:** Balance across tasks/domains using temperature-weighted sampling or curriculum strategies.

1.11.3 Diversity-Promoting Methods

Method	How It Promotes Diversity
Temperature scaling	Higher τ flattens the distribution; more tokens become plausible.
Top- p / Min- p	Adaptive thresholds allow wider sampling when the model is uncertain.
Frequency penalty	Penalizes repeated tokens, forcing lexical variety within a response.
Data deduplication	Removing near-duplicates from training data prevents overfitting to specific patterns.
Multi-domain mixing	Temperature-weighted sampling across domains ensures broad coverage.
Verbalized sampling	Prompt the model to explicitly verbalize a probability distribution over responses [422]. See §7.5.

1.12 Text Generation: Decoding Methods

A trained language model outputs a probability distribution over the vocabulary at each step: $P(x_t|x_{<t})$. The **decoding strategy** determines how we select the next token from this distribution. This choice profoundly affects output quality, diversity, and coherence.

1.12.1 Greedy Decoding

The simplest strategy: always pick the highest-probability token.

$$x_t = \arg \max_{v \in \mathcal{V}} P(v|x_{<t})$$

Intuition: Like always taking the most obvious next word in a sentence. “The capital of France is...” $\rightarrow$ “Paris” (probability 0.92).

Pros: Deterministic, fast, no hyperparameters.

Cons: Produces repetitive, generic text. Misses high-quality sequences where an early low-probability token leads to a globally better output. No diversity.

1.12.2 Beam Search

Maintain B (beam width) partial hypotheses in parallel, expanding each by the top- k tokens and keeping the B highest-scoring complete sequences:

$$\text{score}(y_{1:t}) = \sum_{i=1}^t \log P(y_i | y_{<i})$$

With **length normalization** to avoid favoring short sequences:

$$\text{score}_{\text{norm}}(y) = \frac{1}{|y|^\alpha} \sum_{i=1}^{|y|} \log P(y_i | y_{<i}), \quad \alpha \in [0.6, 1.0]$$

Intuition: Like exploring multiple paths in a maze simultaneously, keeping only the B most promising ones at each junction.

Pros: Finds higher-likelihood sequences than greedy; good for translation and summarization where there’s a single “correct” output.

Cons: Still tends toward generic, repetitive text for open-ended generation; $B \times$ more compute; all beams often converge to similar outputs.

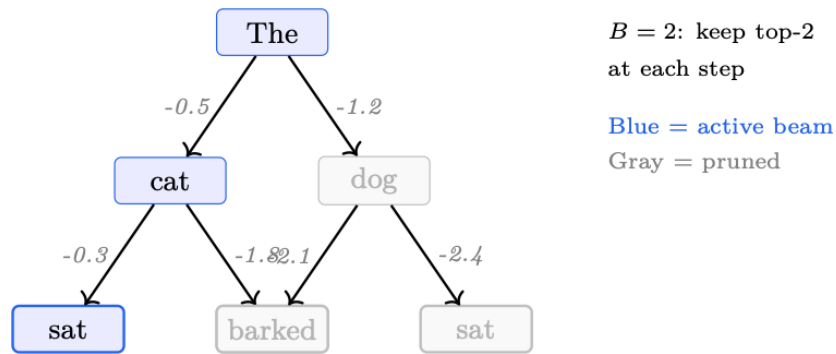


Figure 1.12: Beam search with $B = 2$. At each step, only the 2 highest-scoring partial sequences survive (blue). Lower-scoring alternatives are pruned (gray).

1.12.3 Diverse Beam Search

Standard beam search produces near-duplicate beams. Diverse beam search [360] partitions beams into G groups and adds a **dissimilarity penalty** between groups:

$$\text{score}_g(y_t) = \log P(y_t | y_{<t}) - \lambda \sum_{g' < g} \Delta(y_t, Y_{g'})$$

where Δ measures overlap (e.g., Hamming diversity) with tokens already selected by earlier groups, and λ controls diversity strength.

Intuition: Like forcing a brainstorming group to generate different ideas — each subgroup is penalized for repeating what earlier subgroups said.

Pros: Produces genuinely different candidate sequences; useful for reranking pipelines.

Cons: Diversity penalty can degrade individual beam quality; more hyperparameters (G, λ).

1.12.4 Top- k Sampling

Sample from only the k most probable tokens, redistributing probability mass:

$$P'(v|x_{<t}) = \begin{cases} \frac{P(v|x_{<t})}{\sum_{v' \in \text{Top-}k} P(v'|x_{<t})} & \text{if } v \in \text{Top-}k \\ 0 & \text{otherwise} \end{cases}$$

Intuition: After “The cat sat on the...”, only consider the top k plausible continuations (“mat”, “floor”, “couch”, ...) and ignore extremely unlikely ones (“quantum”, “archipelago”).

Pros: Removes tail noise; simple to implement.

Cons: Fixed k is too restrictive for peaked distributions (wastes probability mass) and too permissive for flat distributions (lets in garbage tokens).

1.12.5 Top- p (Nucleus) Sampling

Sample from the smallest set of tokens whose cumulative probability exceeds p :

$$\text{Top-}p = \min \left\{ S \subseteq \mathcal{V} : \sum_{v \in S} P(v|x_{<t}) \geq p \right\}$$

where tokens are sorted by descending probability and added until the threshold p is reached.

Intuition: Adaptively resize the candidate pool. If the model is confident (“Paris” at 95%), the nucleus is tiny. If uncertain (“The movie was...”), the nucleus expands to include many plausible adjectives.

Pros: Adapts to distribution shape; widely used default ($p = 0.9$ – 0.95).

Cons: Still includes some low-quality tokens at the tail of the nucleus; the threshold is a single global hyperparameter.

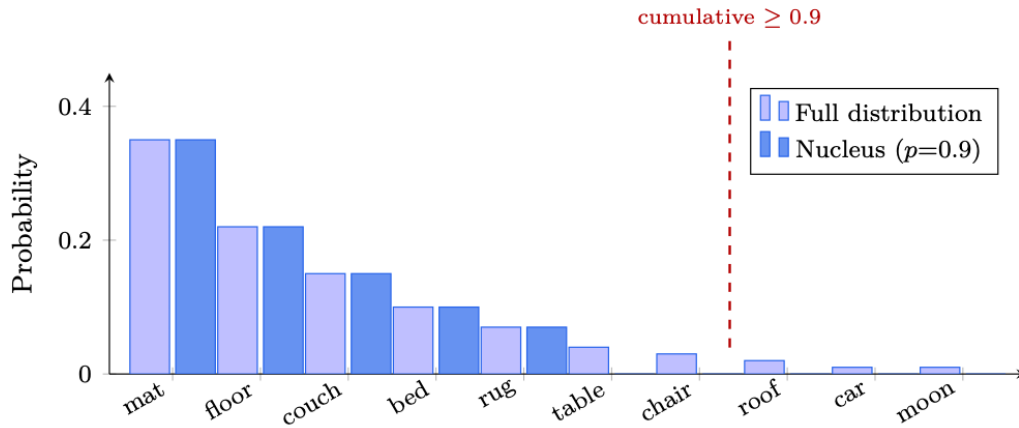


Figure 1.13: Top- p (nucleus) sampling: tokens are sorted by probability and included until cumulative mass reaches $p = 0.9$. The nucleus (dark blue) adapts its size to the distribution shape — here 5 tokens suffice.

Top- k vs. Top- p

Consider predicting the next word:

- After “2 + 2 =”: distribution is peaked — top-1 token (“4”) has 99% mass. Top- $k=50$ wastefully considers 49 wrong answers. Top- $p=0.9$ correctly picks just “4”.
- After “I enjoy eating”: distribution is flat — many foods are plausible. Top- $k=5$ is too restrictive. Top- $p=0.9$ might include 50+ tokens, matching the actual uncertainty.

Top- p adapts; top- k doesn’t. In practice, both are often combined: sample from top- p intersected

with top- k .

1.12.6 Min- p Sampling

A recent alternative that sets a **relative** probability floor [266]:

$$\text{Min-}p = \left\{ v \in \mathcal{V} : P(v|x_{<t}) \geq p_{\min} \cdot \max_{v'} P(v'|x_{<t}) \right\}$$

Only tokens with probability at least $p_{\min}$ times the top token’s probability are kept.

Intuition: “Only consider tokens that are at least 10% as likely as the best token.” If the top token has probability 0.8, only tokens above 0.08 survive. If the top token has probability 0.05 (very uncertain), tokens above 0.005 survive — naturally expanding the pool.

Pros: Scales naturally with model confidence; fewer degenerate samples than top- p on peaked distributions; single intuitive parameter.

Cons: Newer, less battle-tested; not yet standard in all inference frameworks.

1.12.7 Temperature Scaling

Before applying any sampling strategy, logits are divided by temperature T :

$$P_T(v|x_{<t}) = \frac{\exp(z_v/T)}{\sum_{v'} \exp(z_{v'}/T)}$$

- $T < 1$: Sharpens distribution $\rightarrow$ more deterministic, focused outputs.
- $T = 1$: Unmodified model distribution.
- $T > 1$: Flattens distribution $\rightarrow$ more random, creative outputs.
- $T \rightarrow 0$: Becomes greedy decoding. $T \rightarrow \infty$: Becomes uniform sampling.

Common settings: $T = 0.7$ for factual tasks, $T = 1.0$ – 1.2 for creative writing, $T = 0.0$ (greedy) for code/math.

1.12.8 Contrastive Decoding

Contrastive decoding [210] exploits the difference between a strong model (expert) and a weak model (amateur) to amplify the expert’s unique knowledge:

$$x_t = \arg \max_{v \in \mathcal{V}(x_{<t})} [\log P_{\text{expert}}(v|x_{<t}) - \log P_{\text{amateur}}(v|x_{<t})]$$

where $\mathcal{V}(x_{<t}) = \{v : P_{\text{expert}}(v|x_{<t}) \geq \alpha \cdot \max_{v'} P_{\text{expert}}(v'|x_{<t})\}$ is an adaptive plausibility constraint.

Intuition: The amateur model captures generic, obvious patterns (common words, repetition). Subtracting its log-probabilities removes this “generic signal,” leaving the expert’s distinctive knowledge and reasoning. Like removing background noise from a recording to hear the signal.

Pros: Reduces repetition and generic phrasing; improves factuality and coherence without additional training; works with any model pair.

Cons: Requires running two models ($2\times$ compute); sensitive to amateur model choice; the plausibility threshold α needs tuning.

1.12.9 Repetition Penalties

Orthogonal to the sampling strategy, repetition penalties discourage the model from repeating tokens. Given the raw logit z_v for token v (i.e., the unnormalized score output by the LM head *before* softmax), the penalized logit is:

$$z'_v = \begin{cases} z_v/\theta & \text{if } v \in \text{generated tokens and } z_v > 0 \\ z_v \cdot \theta & \text{if } v \in \text{generated tokens and } z_v < 0 \end{cases}$$

where $\theta > 1$ is the penalty factor (typically 1.1–1.3). In both cases, the effect is to push the logit toward zero—reducing the probability of previously generated tokens. Frequency and presence penalties are simpler additive variants used by OpenAI APIs:

$$z'_v = z_v - \alpha \cdot \text{count}(v) - \beta \cdot \mathbf{1}[v \in \text{generated}]$$

where α is the frequency penalty (proportional to how many times v appeared) and β is the presence penalty (flat penalty for any prior occurrence).

1.12.10 Practical Comparison

Table 1.15: Decoding method comparison for LLM text generation.

Method	Deterministic	Diversity	Quality	Best For
Greedy	Yes	None	Medium	Code, factual QA
Beam Search ($B=4-8$)	Yes	Low	High (narrow)	Translation, summarization
Diverse Beam Search	Yes	Medium	High	Candidate generation for reranking
Top- k ($k=50$)	No	Medium	Medium	General-purpose generation
Top- p ($p=0.9$)	No	Adaptive	High	Default for open-ended tasks
Min- p ($p_{\min}=0.1$)	No	Adaptive	High	Robust alternative to top- p
Contrastive	Yes	Low	Very High	Factual, coherent long-form

Decoding in Practice: “Once upon a time”

Given the prompt “Once upon a time,”:

- **Greedy:** “there was a young girl who lived in a small village...” (generic fairy tale)
- **Top- $p=0.9$, $T=1.0$:** “the rivers ran backwards and the fish learned to fly...” (creative, surprising)
- **Top- $p=0.9$, $T=0.3$:** “there was a kingdom ruled by a wise and just king...” (coherent, conventional)
- **Contrastive:** “in the amber-lit corridors of a collapsing star, two minds argued about the nature of time...” (distinctive, avoids clichés)

Same model, same prompt — decoding strategy determines the character of the output.

1.12.11 Constrained Decoding (Structured Generation)

All methods above sample from the *full* vocabulary at each step. **Constrained decoding** restricts the set of allowed tokens so that the output is *guaranteed* to conform to a formal grammar—typically a JSON schema, regex, or context-free grammar (CFG).

Core mechanism. At each decoding step t , a **token mask** $M_t \subseteq \mathcal{V}$ is computed from the current parser state. Only tokens in M_t receive their original logits; all others are set to $-\infty$ before softmax:

$$P'(v|x_{<t}) = \begin{cases} P(v|x_{<t})/Z & \text{if } v \in M_t \\ 0 & \text{otherwise} \end{cases}$$

where $Z = \sum_{v \in M_t} P(v|x_{<t})$ renormalizes. Because the mask changes every step (it depends on what has been generated so far), the constraint is enforced *incrementally*—the model cannot produce an invalid prefix at any point.

From schema to mask. The compilation pipeline is:

$$\text{JSON Schema} \xrightarrow{\text{compile}} \text{Regex} \xrightarrow{\text{compile}} \text{FSM (DFA)} \xrightarrow{\text{index}} \text{Token Mask per State}$$

The FSM states correspond to positions in the regex. For each state, all vocabulary tokens that would keep the string in the language are precomputed into an index (a one-time cost per schema). At runtime, looking up the mask is an $O(1)$ table access—adding negligible latency to each decoding step.

Key libraries.

- **Outlines** [378]: Compiles JSON schemas and regexes into interleaved FSM-guided generation. Supports any model with a logits interface.
- **lm-format-enforcer**¹: Similar FSM approach with a focus on integration with serving frameworks (vLLM, TGI).
- **Guidance**² (Microsoft): Interleaves constrained generation with control flow (loops, conditions), enabling complex structured outputs beyond flat schemas.
- **XGrammar** [77]: Pushdown-automaton-based engine supporting full context-free grammars (not just regular languages), used in MLC-LLM and vLLM for grammar-mode decoding.

Trade-offs. Constrained decoding *guarantees* syntactic validity—no post-hoc parsing failures, no retries. However:

- **Semantic quality:** Forcing structure can degrade content quality if the model’s probability mass for the “correct” answer lies outside the grammar. In practice this is rare for well-trained models on well-designed schemas.
- **Compilation cost:** The FSM index must be built per schema. For complex schemas this can take 1–5 s, but it is amortized over all requests using that schema.
- **Grammar coverage:** Regex/FSM handles JSON, YAML, SQL fragments, and most structured formats. Full CFGs (via XGrammar or LALR parsers) cover languages like Python or XML.

¹<https://github.com/noamgat/lm-format-enforcer>

²<https://github.com/guidance-ai/guidance>

When to Use Constrained Decoding

Use constrained decoding whenever the consumer of the model’s output is a program rather than a human. Tool-calling agents, API backends, and data-extraction pipelines all benefit from *guaranteed* valid structure. For free-form prose or creative text, unconstrained sampling remains appropriate.

1.13 Prompt Engineering

Prompt engineering is the discipline of designing inputs to LLMs that reliably elicit desired behaviour—without changing model weights. While fine-tuning modifies the model, prompt engineering exploits the model’s *existing* capabilities through careful framing, examples, and structure. It is the fastest, cheapest, and most accessible lever for improving LLM outputs, and remains essential even when using fine-tuned models.

1.13.1 In-Context Learning (ICL)

In-context learning [32] is the remarkable ability of large language models to learn tasks at inference time purely from examples provided in the prompt—with no gradient updates. The model implicitly infers the task from the pattern of input–output pairs and generalizes to new inputs.

Why In-Context Learning Works

- **Implicit Bayesian inference:** The model has seen millions of tasks during pretraining. The prompt examples *locate* the relevant task in the model’s learned distribution [396].
- **Induction heads:** Specific attention heads learn to copy patterns (“A is to B as C is to ”), enabling in-context generalization [273].
- **Task vectors:** ICL creates implicit task representations in the residual stream that steer generation toward the demonstrated format and content [354].

Scaling behaviour. ICL emerges primarily in models above ~1B parameters and improves logarithmically with model scale [32]. Smaller models can memorize examples but struggle to generalize to novel inputs within the same context window.

1.13.2 Zero-Shot Prompting

Zero-shot prompting provides *no* examples—only a task description or instruction. The model must rely entirely on its pretrained knowledge and instruction-tuning to produce the correct format and content.

Zero-Shot Classification

Classify the following movie review **as** POSITIVE **or** NEGATIVE.

Review: "The cinematography was breathtaking but the plot felt rushed and predictable."

Sentiment:

When zero-shot works well:

- Tasks the model has seen extensively during pretraining/SFT (translation, summarization, sentiment)
- Well-specified instructions with unambiguous output format
- Instruction-tuned models (e.g., ChatGPT, Claude, Llama-3-Instruct) significantly outperform base models at zero-shot tasks [280]

When zero-shot fails: Novel formats, domain-specific labeling schemes, or ambiguous tasks where the model cannot infer your exact requirements from the instruction alone.

1.13.3 Few-Shot Prompting

Few-shot prompting [32] provides k input–output examples (“shots”) before the actual query. This is the most common form of in-context learning and remains one of the most effective prompting strategies.

Few-Shot Named Entity Recognition

```
Extract named entities from the text. Format: [ENTITY](TYPE)

Text: "Apple released the iPhone 15 in Cupertino."
Entities: [Apple](ORG), [iPhone 15](PRODUCT), [Cupertino](LOC)

Text: "Elon Musk announced Tesla’s new factory in Berlin."
Entities: [Elon Musk](PER), [Tesla](ORG), [Berlin](LOC)

Text: "OpenAI partnered with Microsoft to deploy GPT-4."
Entities:
```

Key design principles for few-shot examples:

1. **Diversity:** Cover the range of expected inputs (different lengths, edge cases, categories).
2. **Ordering:** Place harder or more representative examples last (recency bias) [241].
3. **Label balance:** If classifying, include examples from all classes to avoid majority-class bias.
4. **Format consistency:** Every example must follow the *exact* same structure. The model mimics the pattern.
5. **Relevance:** Use examples semantically similar to the target query for best results [223].

How many shots? Performance typically improves from 0 to 4–8 examples, then plateaus. Beyond ~20 examples, gains are marginal and you risk filling the context window. Min et al. [257] showed that the *format* and *label space* of examples matter more than label correctness—even random labels help (though correct labels help more).

1.13.4 Instruction-Following Prompts

Instruction-tuned models respond best to clear, structured instructions. The key insight: treat the prompt as a *specification*, not a suggestion.

Anatomy of an Effective Instruction Prompt

1. **Role/Persona:** Define who the model is (“You are a senior data scientist...”)
2. **Task:** What to do, stated clearly and unambiguously
3. **Context:** Background information the model needs
4. **Constraints:** Length limits, tone, what to avoid, output format
5. **Examples** (optional): Show the desired output format
6. **Input:** The actual data to process

Instruction Prompt with Constraints

```

Role: You are a medical literature reviewer.

Task: Summarize the following research abstract for a
general audience.

Constraints:
- Maximum 3 sentences
- No jargon (explain any technical terms)
- Include the key finding and its clinical implication
- Do NOT speculate beyond what the abstract states

Abstract: [...]

```

System prompts vs. user prompts. Modern chat APIs separate the *system* prompt (persistent instructions, role definition) from the *user* message (per-turn input). System prompts are processed with higher attention priority in most models and provide a natural place for role definitions, constraints, and output format specifications [274].

1.13.5 Structured Output Prompts (JSON/XML)

For programmatic use, the most critical prompting technique is enforcing structured output—particularly JSON.

JSON Output Prompt

```

Extract the following information from the customer email.
Respond ONLY with valid JSON, no other text.

Schema:
{
  "intent": "refund|complaint|question|praise",
  "urgency": "low|medium|high",
  "product_mentioned": "string or null",
  "summary": "one sentence summary"
}

Email: [...]

```

Techniques for reliable structured output:

- **Schema-first:** Show the exact JSON schema *before* the input. The model treats it as a template.
- **Constrained decoding:** Use grammar-based sampling (e.g., Outlines [378], Guidance) to guarantee syntactically valid JSON at the token level.
- **XML tags:** For nested or multi-part outputs, XML tags (e.g., `<thinking>...</thinking>`) provide unambiguous delimiters that models follow reliably.
- **Pydantic/TypeScript types:** Providing type definitions helps models understand field constraints (OpenAI's function calling uses JSON Schema internally).

JSON in Prompts — Common Pitfalls

- Models may add markdown fences (`"`json ... "``) — instruct explicitly to output raw JSON.
- Nested objects and arrays increase hallucination risk — flatten schemas where possible.

- Enum fields (fixed choices) are much more reliable than free-text fields.
- Always validate outputs programmatically; no prompt guarantees 100% compliance without constrained decoding.

JSON Prompting: Structuring the Input. A distinct but complementary technique is *JSON prompting*—formatting the prompt *itself* as JSON rather than natural language. This exploits the model's extensive pre-training on structured data (APIs, configs, code) to improve instruction adherence, reduce ambiguity, and enable deterministic parsing of multi-field requests.

JSON Prompting with System Prompt

```
=== SYSTEM ===
You are a senior code reviewer. Analyze code for bugs,
security issues, and style violations. Always respond
in the JSON schema provided.

=== USER (JSON prompt) ===
{
  "task": "code_review",
  "language": "python",
  "severity_filter": "high",
  "code": "def login(user, pw):\n    query = ...",
  "output_schema": {
    "issues": [{
      "line": "int",
      "severity": "critical|high|medium|low",
      "category": "security|bug|style|performance",
      "description": "string",
      "fix": "string"
    }],
    "overall_risk": "critical|high|medium|low"
  }
}
```

Why JSON prompting works:

- **Unambiguous field boundaries:** No confusion about where one instruction ends and another begins.
- **Typed constraints:** Fields like "severity_filter": "high" are clearer than “only show high severity issues.”
- **Schema-as-contract:** Including output_schema in the input mirrors API design patterns the model has seen extensively during pre-training.
- **System prompt still essential:** The system message provides *role*, *tone*, and *behavioral constraints* that don't fit naturally in a JSON payload.

1.13.6 Chain-of-Thought (CoT) Prompting

Chain-of-thought prompting [374] asks the model to produce intermediate reasoning steps before giving a final answer. This simple technique dramatically improves performance on tasks requiring multi-step reasoning: arithmetic, logic, commonsense inference, and code generation.

Why CoT works:

- **Serializes computation:** Transformers have fixed depth but variable-length generation. CoT converts parallel (hard) problems into sequential (easy) steps, effectively increasing the model's computational budget.

- **Reduces compounding errors:** Each step is a simpler sub-problem with lower per-step error rate.
- **Exposes intermediate state:** Makes reasoning auditable and debuggable.

Chain-of-Thought Variants

Method	Description
Zero-shot CoT [189]	Append “Let’s think step by step” to any prompt
Few-shot CoT [374]	Provide examples with explicit reasoning chains
Self-Consistency [367]	Sample N CoT paths; majority-vote the final answer
Tree of Thoughts [407]	Explore multiple reasoning branches with backtracking
Plan-and-Solve [364]	First plan the steps; then execute each step
ReAct [408]	Interleave Reasoning and Acting (tool use)

Zero-Shot Chain-of-Thought

Q: A store has 45 apples. They sell $\frac{3}{5}$ of them in the morning and $\frac{1}{3}$ of the remaining in the afternoon. How many apples are left?

Let’s think step by step.

A: Morning sales: $45 * \frac{3}{5} = 27$ apples sold.
 Remaining after morning: $45 - 27 = 18$.
 Afternoon sales: $18 * \frac{1}{3} = 6$ apples sold.
 Remaining: $18 - 6 = 12$ apples.

Self-Consistency. Wang et al. [367] showed that sampling multiple chain-of-thought reasoning paths and taking a majority vote over final answers significantly outperforms single-path CoT. The intuition: correct reasoning paths tend to converge on the same answer, while errors are typically idiosyncratic. This trades compute (generating N samples) for accuracy—practical when latency is less important than correctness.

When CoT hurts. CoT is not universally beneficial. For simple tasks (single-step classification, retrieval, formatting), CoT adds unnecessary tokens, increases latency, and can even introduce errors through overthinking. Use CoT selectively for tasks where you expect multi-step reasoning to be required.

1.13.7 Advanced Prompting Techniques

Retrieval-Augmented Generation (RAG). Rather than relying solely on the model’s parametric memory, RAG [208] retrieves relevant documents and includes them in the prompt:

```
Context (retrieved): [document chunks]
Question: [user query]
Answer based ONLY on the provided context.
```

This grounds the model’s responses in verifiable sources and dramatically reduces hallucinations for knowledge-intensive tasks.

Prompt Chaining and Decomposition. Complex tasks benefit from being broken into a pipeline of simpler prompts, where the output of one becomes the input to the next:

1. *Extract* key facts from document
2. *Reason* over extracted facts

3. *Format* final answer

Each step can use a different prompt template, model, or temperature setting. This is more controllable than a single monolithic prompt and enables targeted debugging.

Constitutional AI / Self-Critique. Bai et al. [16] introduce prompts that ask the model to critique and revise its own output against a set of principles:

```
[Generate initial response]
Critique: Does this response violate any of the following
principles? [list principles]
Revision: Rewrite the response addressing the critique.
```

Meta-Prompting and Prompt Optimization. Rather than hand-crafting prompts, recent work automates prompt design:

- **APE** [441]: Uses an LLM to generate and score candidate prompts automatically.
- **DSPy** [183]: Compiles declarative task descriptions into optimized prompt pipelines with learned few-shot examples.
- **OPRO** [402]: Treats prompt optimization as an optimization problem, using an LLM as the optimizer.

Attentive Reasoning Queries (ARQ). ARQ [403] addresses a fundamental weakness of standard prompting: as context length grows, models increasingly “lose” critical information in the middle of the prompt (the *lost-in-the-middle* effect). ARQ mitigates this by decomposing a complex query into multiple focused sub-queries, each designed to direct the model’s attention to a specific part of the context:

1. **Query decomposition:** Break the user question into atomic sub-questions that each target a narrow aspect.
2. **Attentive retrieval:** For each sub-query, retrieve or highlight only the relevant context slice—forcing the model to attend to it.
3. **Aggregation:** Combine sub-answers into a coherent final response.

This is particularly effective for long-document QA, multi-hop reasoning over large retrieval sets, and agentic tasks where the context window contains many tool outputs. ARQ can be seen as a structured form of chain-of-thought that explicitly manages *where* the model looks, not just *how* it reasons.

1.13.8 Best Practices: Crafting Effective Prompts

Based on empirical findings across the literature and practitioner experience, the following principles reliably improve prompt quality:

The Prompt Engineering Checklist

1. **Be specific and unambiguous:** Replace “summarize this” with “summarize in 2–3 bullet points, each under 20 words, focusing on actionable findings.”
2. **Show, don’t tell:** One good example is worth 100 words of instruction. When in doubt, add a few-shot example.
3. **Define the output format explicitly:** Specify JSON schema, bullet points, table format, or exact delimiters. Never leave format to interpretation.

4. **Use delimiters for input data:** Wrap user inputs in clear delimiters (`"""`, `<input>...</input>`, `--`) to separate instructions from data.
5. **Assign a role:** “You are a [domain expert] who [specific behaviour]” primes relevant knowledge and tone.
6. **Specify what NOT to do:** Negative constraints (“do not explain your reasoning”, “never output more than 5 items”) are often more effective than positive ones.
7. **Add chain-of-thought for reasoning tasks:** Append “Think step by step” or provide worked examples for math, logic, or multi-hop questions.
8. **Control temperature appropriately:** Use $T \approx 0$ for factual/deterministic tasks; $T \approx 0.7$ – 1.0 for creative/diverse outputs.
9. **Iterate empirically:** Treat prompts as code—version them, A/B test them, and measure performance on representative eval sets.
10. **Leverage recency bias:** Place the most critical instructions and examples at the *end* of the prompt (closest to the generation point).

Table 1.16: Common prompting failure modes and solutions.

Failure Mode	Symptom	Solution
Instruction amnesia	Model ignores constraints in long prompts	Move constraints to end; repeat key rules; use system prompt
Format drift	Output starts correct but degrades over long generations	Use constrained decoding; break into shorter chained prompts
Sycophancy	Model agrees with incorrect premises in the prompt	Add “challenge assumptions if incorrect”; use system-level instruction
Hallucinated details	Model invents facts not in provided context	Add “if unknown, say I don’t know”; use RAG with source attribution
Refusal over-triggering	Model refuses benign requests due to safety training	Rephrase to clarify legitimate intent; provide explicit context for why the request is appropriate

The Prompt Engineering Mindset

Think of prompt engineering as *programming in natural language*. The model is a powerful but literal interpreter—it will do exactly what you ask, interpreted in the most likely way given its training distribution. Common principles from software engineering apply:

- **DRY (Don’t Repeat Yourself):** Unless fighting attention decay in long contexts
- **Separation of concerns:** Different prompt sections for role, constraints, examples, and input
- **Test-driven development:** Define expected outputs before writing the prompt
- **Version control:** Track prompt iterations and their eval scores
- **Modularity:** Build reusable prompt templates; parameterize variable parts

When prompting fails to achieve the desired quality after systematic iteration, that is the signal to move to fine-tuning (SFT) or reinforcement learning (RLHF/DPO).

1.14 Model Compression Methods

Model compression reduces model size and inference cost while preserving quality. Three main approaches: quantization (reduce precision), pruning (remove parameters), and distillation (train a smaller model to mimic a larger one).

1.14.1 Quantization

Quantization reduces model size and inference cost by representing weights (and optionally activations) in lower-precision formats. The core trade-off is compression ratio versus quality degradation.

Quantization Overview

Quantization reduces the numerical precision of model weights (and optionally activations) from FP32/BF16 to lower-bit formats:

$$x_q = \text{round}\left(\frac{x - z}{s}\right), \quad x_{\text{dequant}} = s \cdot x_q + z$$

where s is the scale factor and z is the zero-point.

Table 1.17: Quantization methods for LLMs.

Method	Bits	Type	Key Idea
GPTQ [99]	4-bit	PTQ, weight-only	Layer-wise quantization minimizing $\ WX - \hat{W}X\ ^2$ via optimal brain surgeon.
AWQ [217]	4-bit	PTQ, weight-only	Protects salient weights (those with large activations). 1% of weights carry 99% importance.
GGUF [109]	2–8 bit	PTQ, weight-only	CPU-optimized format (llama.cpp). Per-block quantization with multiple types.
FP8 (E4M3)	8-bit	Training + inference	Native H100 support. 2× throughput vs BF16.
SmoothQuant [392]	W8A8	PTQ, weight+act.	Smooths activation outliers into weights before quantization. Enables INT8 GEMM.
QAT [234]	4-bit	QAT	Trains with simulated quantization. Highest quality but expensive.
AQLM [83]	2-bit	PTQ, additive codes	Extreme compression via learned additive quantization codebooks.

When to Quantize

- **Inference serving:** Always quantize. W4A16 (4-bit weights, BF16 activations) is the sweet spot — 2× memory savings, <1% quality loss for 70B+ models.
- **Training:** FP8 on H100 gives 2× throughput with minimal quality loss. BF16 is still the default for smaller models.
- **Edge deployment:** GGUF Q4_K_M for local inference on consumer hardware.
- **RLHF:** Quantize the frozen models (reference, reward model) to INT8/FP8. Keep the policy in BF16 for training precision.

1.14.2 Pruning

Why Prune? Modern LLMs contain billions of parameters, yet empirical studies consistently show that a large fraction of these weights contribute minimally to model outputs. Pruning exploits this

over-parameterization: by removing redundant weights, we reduce **memory footprint** (enabling deployment on smaller GPUs or edge devices), **inference latency** (fewer multiply-accumulate operations per forward pass), and **serving cost** (higher throughput per dollar). Unlike quantization, which reduces the precision of all weights uniformly, pruning selectively eliminates weights—enabling multiplicative savings when combined with quantization (e.g., a 50% sparse, 4-bit model uses $4\times$ less memory than the dense BF16 baseline). The challenge is achieving high sparsity without degrading generation quality, which has driven the development of principled one-shot methods that require no retraining.

Pruning Methods

- **Unstructured pruning:** Zero out individual weights below a threshold. High sparsity (50–90%) possible. Requires sparse GEMM kernels (2:4 on A100/H100).
- **Structured pruning:** Remove entire attention heads, layers, or FFN neurons. Directly reduces FLOPS without specialized kernels.
- **SparseGPT [98]:** One-shot pruning using approximate inverse Hessian. 50% unstructured sparsity with minimal quality loss on 175B models.
- **Wanda [345]:** Prune by $|w| \times \|x\|$ (weight magnitude times input activation norm). No calibration data needed. Competitive with SparseGPT.

NVIDIA 2:4 Structured Sparsity

A100/H100 Tensor Cores natively support 2:4 sparsity: out of every 4 elements, at most 2 are non-zero. This gives exactly $2\times$ speedup on supported operations with hardware acceleration. The constraint: you must achieve *exactly* 50% sparsity in this specific pattern, which limits flexibility compared to arbitrary sparsity.

1.14.3 Knowledge Distillation

Knowledge distillation [139] transfers the learned behaviour of a large *teacher* model into a smaller, cheaper *student* model. The core idea is that the teacher’s output distribution over tokens carries far richer signal than ground-truth hard labels alone — revealing inter-class similarities, calibration, and uncertainty that the student can exploit.

Temperature-Scaled Softmax. To expose the “dark knowledge” in the teacher’s logits we soften the distribution with a temperature $T > 1$:

$$p_i^{(T)} = \frac{\exp(z_i/T)}{\sum_j \exp(z_j/T)}$$

At high temperature the probability mass spreads across more tokens, making near-miss alternatives visible. During training the same temperature is applied to the student; at inference the student uses $T = 1$.

General Distillation Loss.

$$\mathcal{L}_{\text{distill}} = \alpha T^2 \cdot \text{KL}(P_{\text{teacher}}^{(T)} \parallel P_{\text{student}}^{(T)}) + (1 - \alpha) \cdot \mathcal{L}_{\text{CE}}(y, P_{\text{student}}^{(1)})$$

The T^2 factor compensates for the reduced gradient magnitude of softened distributions. Typical values: $T \in [2, 20]$, $\alpha \in [0.5, 0.9]$ (more weight on KL when teacher quality is high).

Offline (White-Box) Distillation. The teacher’s full logit vector (or top- k logits for storage efficiency) is recorded for each training token. The student minimises the KL divergence against these stored distributions. This is the most data-efficient paradigm when teacher access is unrestricted.

Motivation: Decouple teacher inference from student training — run the teacher once on high-end hardware, then train many students cheaply.

Table 1.18: Knowledge distillation paradigms for LLMs.

Paradigm	Mechanism	Pros	Cons
Offline / White-box	Teacher logits pre-computed; student trains on full distributions	Full distribution signal; one-time teacher cost	Stale data; storage heavy
Online / Co-training	Teacher generates on-the-fly; student sees fresh logits	Adapts to student weaknesses	2× compute; complex infra
Black-box (API)	Only teacher <i>text</i> outputs available (no logits)	Works with proprietary models	Loses dark knowledge; SFT-like
Self-distillation	Model distills into a smaller version of itself	No separate teacher needed	Teacher = student family; ceiling

Pros: Deterministic, reproducible; teacher cost is amortised; full distributional signal.

Cons: Requires storing $|V|$ -dimensional vectors per token (mitigated by top- k pruning); teacher cannot adapt to student failures.

Online (Co-Training) Distillation. Teacher and student are run jointly: the teacher generates logits for the student’s current training batch.

Motivation: Let the teacher focus on inputs where the student currently struggles (curriculum-like).

Pros: Freshness; can use student-generated inputs for on-policy distillation.

Cons: Double the GPU cost; synchronisation complexity; harder to scale.

Black-Box (API) Distillation. When only text outputs are available (e.g. distilling from a proprietary API), the student is trained via SFT on the teacher’s generations, optionally augmented with chain-of-thought traces.

Motivation: Practical reality — most frontier models do not expose logits.

Pros: Simple pipeline; works with any model behind an API.

Cons: No soft-label signal; prone to hallucination amplification; effectively supervised fine-tuning.

Self-Distillation. A model distils from a larger version within the same architecture family (e.g. Llama-3 70B → 8B) or from its own checkpoints during training.

Motivation: Avoid training a separate teacher; leverage the model’s own capacity at different scales.

Pros: Architecture compatibility; no external dependency.

Cons: Teacher ceiling equals model ceiling; cannot introduce genuinely new knowledge.

Dark Knowledge

Consider a language model predicting the next word after “The capital of France is”. Hard labels say only “Paris” is correct. But the teacher’s soft distribution might assign 5% to “Lyon”, 2% to “Marseille”, and near-zero to “banana” — telling the student *which errors are reasonable*, which dramatically improves calibration and generalisation.

Practical Considerations for LLM Distillation.

- **Sequence-level vs. token-level:** Token-level KL is standard; sequence-level distillation (minimising KL over full sequences) better captures long-range coherence but is harder to optimise.
- **Layer-wise hints:** Matching intermediate representations (attention maps, hidden states) provides additional learning signal — especially useful when student architecture differs.
- **Data selection:** Distillation data quality matters; curating diverse, hard examples yields better students than random sampling.
- **Student capacity:** Diminishing returns below $\sim 10\%$ of teacher parameters; at extreme compression, architecture changes (e.g. MoE $\rightarrow$ dense) may be needed.
- **Combining with quantization:** Distillation + 4-bit quantization (e.g. QLoRA-distilled models) achieves near-teacher quality at $20\times$ compression.

Compression Method Comparison – 70B Model

Method	Size	Speed	Quality	Use Case
BF16 (baseline)	140 GB	$1\times$	100%	Training, reference
FP8 (E4M3)	70 GB	$2\times$	99.5%	H100 inference
INT8 (SmoothQuant)	70 GB	$1.8\times$	99%	A100 inference
4-bit (AWQ)	35 GB	$2.5\times$	97–98%	Serving at scale
2-bit (AQLM)	17.5 GB	$3\times$	90–93%	Edge, experimental
Pruned 50% (2:4)	70 GB	$1.8\times$	97%	Structured speedup
Distilled 8B	16 GB	$10\times$	80–85%	Mobile, edge

1.15 Speculative Decoding Methods

Speculative decoding [205] accelerates autoregressive generation by predicting multiple tokens simultaneously, then verifying them in a single forward pass of the target model. It produces **identical output distribution** to standard decoding (no quality loss) while achieving $2\text{--}3\times$ speedup.

1.15.1 Core Principle

Speculative Decoding Framework

1. A fast **draft mechanism** proposes k candidate tokens: $\hat{x}_1, \dots, \hat{x}_k$
2. The large **target model** runs a single forward pass on all k tokens (batched)
3. **Verification:** Accept tokens left-to-right while $P_{\text{target}}(\hat{x}_i) \geq r_i \cdot P_{\text{draft}}(\hat{x}_i)$ (where $r_i \sim U[0, 1]$)
4. On first rejection at position j : resample x_j from an adjusted distribution, discard $\hat{x}_{j+1}, \dots, \hat{x}_k$

Key property: This acceptance/rejection scheme guarantees the final distribution equals P_{target} exactly.

Speedup: If acceptance rate is α , expected tokens per step = $\frac{1-\alpha^{k+1}}{1-\alpha}$. At $\alpha = 0.8$, $k = 5$: expected 3.4 tokens/step vs. 1 for standard decoding.

Table 1.19: Speculative decoding methods supported by modern inference engines.

Method	Draft Source	Speedup	Key Idea
Standard [205]	Small model (1–7B)	2–3×	Separate draft model generates candidates. Simple but requires loading 2 models.
Medusa [35]	Parallel LM heads	2–3×	Add k extra prediction heads to the target model. Each predicts token at position $+1, +2, \dots, +k$.
Eagle [212]	Feature-level	2.5–3.5×	Lightweight decoder generates draft tokens from target model’s hidden states. Higher acceptance than Medusa.
Eagle-2 [212]	Context-aware	3–4×	Dynamic draft tree with confidence-based expansion. State-of-the-art acceptance rates.
N-gram Lookup	N-gram cache	1.5–2×	Match prompt n-grams against previously generated text. Zero cost; great for repetitive outputs.
Lookahead [101]	Jacobi iteration	2–2.5×	Parallel Jacobi decoding with n-gram verification. No draft model; uses target model itself.
Multi-token [111]	Modified arch.	2–3×	Train the model to natively predict multiple tokens per step (Meta’s approach in Llama).

1.15.2 Methods Comparison

1.15.3 Medusa: Multi-Head Speculative Decoding

How Medusa Works

Medusa adds k additional “prediction heads” to the LLM (sharing the same backbone):

- Head 0 (original): predicts token at position $t + 1$ (standard next-token)
- Head 1: predicts token at position $t + 2$ (skipping one)
- Head i : predicts token at position $t + i + 1$
- All heads run in parallel during a single forward pass
- A **tree-structured** verification validates multiple candidate sequences simultaneously

Training: Fine-tune only the Medusa heads (backbone frozen). Cost: ~ 1 epoch on representative data.

Advantage: No separate draft model; heads are tiny (one linear layer each). Memory overhead: $< 1\%$.

1.15.4 Eagle: Feature-Level Drafting

Why Eagle Outperforms Medusa

Medusa’s heads predict independently at each position — they cannot condition on their own previous predictions (token at $t + 2$ doesn’t know what was predicted at $t + 1$). Eagle fixes this with a lightweight autoregressive decoder that operates on the target model’s hidden states:

1. Extract hidden states from the target model’s last layer

2. Feed into a small (1-layer) decoder that autoregressively generates draft tokens conditioned on previous hidden states
3. This captures inter-token dependencies that Medusa misses

Result: Eagle achieves 85–95% acceptance rate vs. Medusa's 60–80%.

1.15.5 N-gram Speculative Decoding

N-gram Lookup Method

The simplest speculative decoding — requires no additional model or training:

1. Maintain a cache of n-grams from the prompt and previously generated text
2. At each step, check if the current context's last $n - 1$ tokens match any cached n-gram
3. If yes: propose the continuation as draft tokens
4. Verify against target model as usual

Best for: Code generation (repetitive patterns), structured outputs (JSON/XML), and prompts with repeated elements. **Cost:** Essentially zero.

1.15.6 Integration with vLLM

```

from vllm import LLM, SamplingParams

# Standard speculative decoding (separate draft model)
llm = LLM(
    model="meta-llama/Llama-3-70B",
    tensor_parallel_size=4,
    speculative_config={
        "model": "meta-llama/Llama-3-8B",
        "num_speculative_tokens": 5,
    },
)

# N-gram speculation (zero-cost, no draft model needed)
llm = LLM(
    model="meta-llama/Llama-3-70B",
    speculative_config={
        "method": "ngram",
        "num_speculative_tokens": 5,
        "prompt_lookup_max": 4, # Match up to 4-grams from prompt
    },
)

# EAGLE-style (feature-level draft, high acceptance rate)
llm = LLM(
    model="meta-llama/Meta-Llama-3-8B-Instruct",
    tensor_parallel_size=4,
    speculative_config={
        "model": "yuhuili/EAGLE-LLaMA3-Instruct-8B",
        "num_speculative_tokens": 2,
        "method": "eagle",
        "draft_tensor_parallel_size": 1,
    },
)

# MLP speculator (IBM-style, lightweight head)
llm = LLM(
    model="meta-llama/Meta-Llama-3.1-70B-Instruct",
    tensor_parallel_size=4,
    speculative_config={
        "model": "ibm-ai-platform/llama3-70b-accelerator",
        "draft_tensor_parallel_size": 1,
    },
)

```

When NOT to Use Speculative Decoding

- **High batch sizes:** At batch ≥ 64 , generation is already compute-efficient. Speculation adds overhead (draft generation + verification) that doesn't pay off.
- **Very different distributions:** If draft model is too dissimilar to target, acceptance rate drops below 50% and speculation is slower than standard decoding.
- **Short outputs:** For < 20 token outputs, the setup cost of speculation exceeds savings.
- **Rule of thumb:** Speculation helps most for latency-sensitive, single-stream generation (chatbots, interactive code completion).

1.16 Hallucination Detection

LLMs generate fluent text that may be factually incorrect—a phenomenon called **hallucination** [164]. This section covers basic detection methods at the model level (without external retrieval or multi-agent verification).

1.16.1 Types of Hallucination

Hallucination Taxonomy

- **Intrinsic:** Contradicts the provided input/context (e.g., summary says the opposite of the source)
- **Extrinsic:** Generates claims that cannot be verified from the input and are factually wrong
- **Faithfulness:** Output diverges from the instruction or specified constraints

1.16.2 Detection Methods (Model-Level)

Table 1.20: Basic hallucination detection methods that operate at the model level.

Method	Mechanism	Signal
Token-level entropy	High entropy at generation time indicates uncertainty [175]	$H(P(x_t)) > \tau$
Sequence log-prob	Low average log-probability of the output suggests confabulation	$\frac{1}{T} \sum_t \log P(x_t)$
Consistency sampling	Generate N responses; low agreement = likely hallucination [249]	Contradiction rate
Semantic entropy	Cluster meanings (not strings); high semantic entropy = uncertain [193]	Cluster diversity
DoLA	Contrast logits between later vs. earlier layers; amplifies factual knowledge [58]	Layer divergence

Semantic Entropy. Kuhn et al. [193] observe that token-level entropy is unreliable (paraphrases have different tokens but same meaning). Instead, they generate multiple responses, cluster them by semantic equivalence (via NLI), and compute entropy over meaning clusters:

$$SE = - \sum_{c \in \text{clusters}} P(c) \log P(c)$$

High SE means the model produces *semantically different* answers—a strong hallucination signal.

SelfCheckGPT. Manakul et al. [249] detect hallucinations by checking self-consistency: generate multiple responses and verify whether claims in the main response are supported by the alternatives. If the model “disagrees with itself,” the claim is likely hallucinated. No external knowledge needed.

DoLA (Decoding by Contrasting Layers). Chuang et al. [58] observe that factual knowledge emerges in later transformer layers while earlier layers retain more generic/uncertain representations. DoLA contrasts the logit distributions between a later (“mature”) layer and an earlier (“premature”) layer at each decoding step:

$$\text{DoLA}(x_t) = \text{softmax}(\log P_{\text{late}}(x_t) - \log P_{\text{early}}(x_t))$$

By amplifying the signal from factual knowledge encoded in deeper layers, DoLA reduces hallucinations at inference time *without any retraining*—requiring only a single additional forward pass through the contrasted layer. It is complementary to sampling-based methods and can be combined with them.

Limitations of Model-Level Detection

These methods detect *uncertainty*, not *incorrectness*. A model can be confidently wrong (low entropy, consistent responses—but factually false). For reliable detection, combine with retrieval-based verification (RAG) or external fact-checking tools.

1.17 LLM Safety and Responsible AI

Safety is not an afterthought—it is an integral part of the LLM training pipeline. This section covers the key dimensions of LLM safety and the mechanisms used to enforce responsible behavior.

1.17.1 Threat Taxonomy

Table 1.21: LLM safety threat categories.

Category	Description and Examples
Harmful content	Generating toxic, violent, or illegal instructions (bioweapons, CSAM)
Bias and discrimination	Perpetuating stereotypes; unfair treatment across demographics [104]
Privacy violations	Leaking PII from training data; memorization attacks [39]
Jailbreaking	Adversarial prompts that bypass safety guardrails [443]
Misinformation	Generating convincing but false claims (hallucination at scale)
Dual-use	Legitimate capabilities (coding, chemistry) weaponized for harm

1.17.2 Safety Training Pipeline

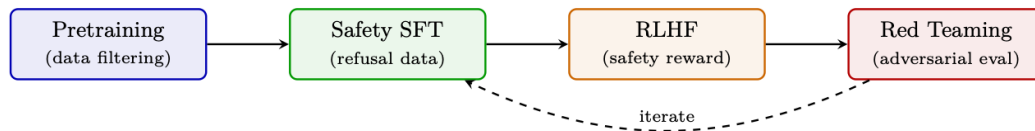


Figure 1.14: Safety is applied at every stage: data filtering in pretraining, refusal examples in SFT, safety-specific reward models in RLHF, and iterative red-teaming.

1.17.3 Key Safety Mechanisms

Safety Techniques

- **Data filtering:** Remove toxic, biased, and PII-containing text from pretraining corpora
- **Safety SFT:** Train on examples of appropriate refusals (“I can’t help with that because. . .”)
- **Constitutional AI** [16]: Self-critique using principles; model revises its own outputs against a constitution of rules
- **Safety reward model:** Separate RM trained on safety-annotated pairs; combined with helpfulness RM during RLHF via weighted sum
- **Guardrails:** Input/output classifiers that block harmful requests/responses at serving time
- **Red teaming** [290]: Systematic adversarial evaluation to find failure modes before deployment

1.17.4 The Helpfulness–Safety Tradeoff

Balancing Helpfulness and Safety

Over-optimizing for safety creates an *over-refusal* problem: the model declines benign requests (e.g., refusing to discuss historical violence in an educational context). The goal is a Pareto-optimal policy that is maximally helpful *within* safety constraints:

$$\max_{\theta} \mathbb{E}[R_{\text{helpful}}] \quad \text{subject to} \quad \mathbb{E}[R_{\text{safety}}] \geq \tau$$

In practice, this is implemented as a weighted reward: $R = \alpha R_{\text{helpful}} + (1 - \alpha) R_{\text{safety}}$ with careful tuning of α (typically 0.6–0.8). Meta’s Llama-3 reports using distinct safety and helpfulness reward models with margin-based weighting [118].

1.17.5 Evaluation

- **Safety benchmarks:** ToxiGen, RealToxicityPrompts, BBQ (bias), CrowS-Pairs
- **Jailbreak robustness:** GCG attacks [443], multi-turn jailbreaks, encoded prompts
- **Over-refusal rate:** Measure false-positive refusals on benign prompts (target <5%)
- **Red team evaluations:** Human adversarial testing with domain experts (biosecurity, cybersecurity)

Safety Is Never Complete

No combination of techniques provides absolute safety. New attack vectors are discovered continuously (multi-modal jailbreaks, fine-tuning attacks that remove safety training, many-shot prompting). Safety requires ongoing monitoring, rapid response to new threats, and defense-in-depth (multiple independent layers).